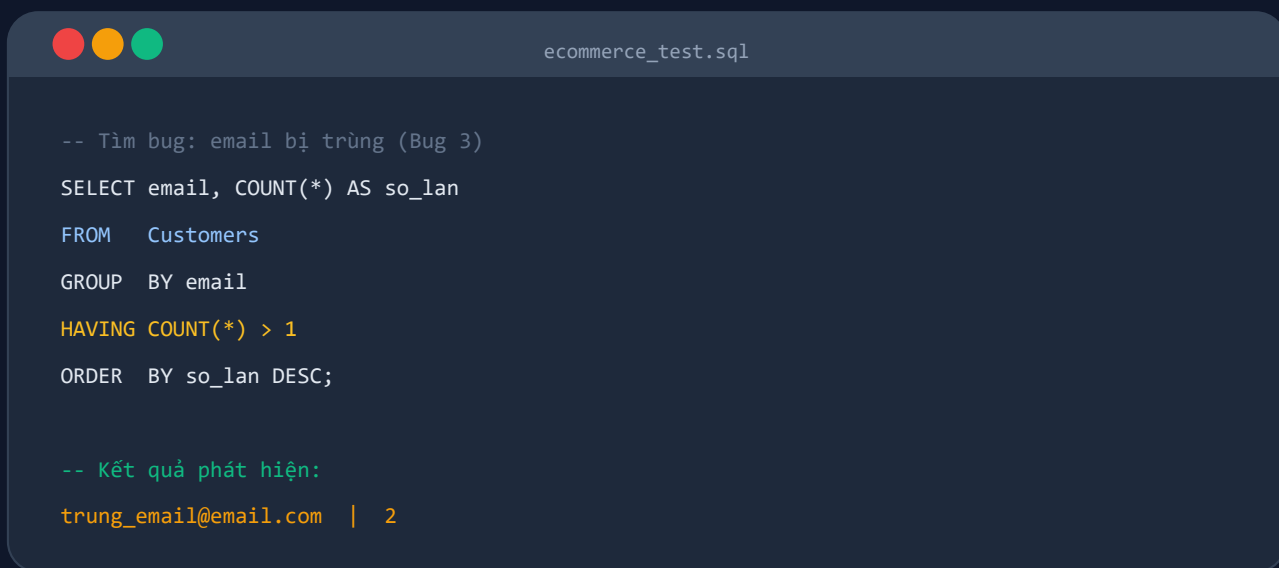


50 CÂU LỆNH SQL

SẴN BUG THỰC CHIẾN

Dành cho QA · Tester · Dev — Dialect MySQL



50

Câu lệnh SQL

6

Nhóm chủ đề

3

Bug cài sẵn

MySQL

Dialect

- Mỗi câu lệnh: tình huống bug thật — SQL — phân tích mệnh đề — kết quả — góc soi lỗi
- Chương 0: script tạo DB, sơ đồ ER, dữ liệu mẫu với 3 bug cố ý cài sẵn để thực hành

Cách dùng cuốn cẩm nang này

Đây không phải tài liệu dạy SQL từ đầu. Nó là một bộ công cụ thực chiến: 50 câu lệnh để một người làm kiểm thử có thể tự mình đi vào database và tìm ra những lỗi mà giao diện không phơi bày. Mỗi câu lệnh được trình bày theo cùng một khuôn để bạn tra cứu nhanh:

- **Dữ liệu trước query** — bảng mẫu cho thấy trạng thái dữ liệu, dòng lỗi highlight đỏ.
- **Câu lệnh SQL** — sẵn sàng copy, viết theo dialect MySQL.
- **Phân tích từng mệnh đề** — mỗi từ khóa SQL được giải thích riêng.
- **Kết quả sau query** — bảng output sau khi chạy lệnh trên dữ liệu mẫu.
- **Góc soi lỗi của Tester** — cạm bẫy, hiểu lầm và bước tiếp theo.

ĐỌC TRƯỚC KHI CHẠY

Mọi câu lệnh trong sách đều là truy vấn ĐỌC (SELECT) — không sửa, không xóa dữ liệu. Dù vậy, hãy luôn chạy trên bản sao test/staging. Tên bảng/cột dùng schema ecommerce_test ở Chương 0 — đổi cho khớp schema thật của bạn.

NGUYÊN TẮC XUYỀN SUỐT

Săn bug bằng SQL thực chất là đi tìm những điều LẪI RA LUÔN ĐÚNG nhưng dữ liệu lại nói ngược lại: tồn kho không thể âm, tổng dòng con phải bằng header, email trong hệ thống phải là duy nhất. Với mỗi bất biến đó, bạn viết một câu lệnh tìm các bản ghi VI PHẠM. Cuốn sách này là 50 ví dụ của đúng một tư duy đó.

Chương 0 — Chuẩn bị môi trường thực hành

Tất cả 50 câu lệnh trong sách đều chạy được ngay trên cơ sở dữ liệu **ecommerce_test** dưới đây. Hệ thống mô phỏng một sàn thương mại điện tử nhỏ gồm 4 bảng, với **3 bug được cố ý cài cắm** để bạn thực hành phát hiện.

1.1 Sơ đồ quan hệ 4 bảng (Entity Relationship)

Customers	Products
PK customer_id customer_name email membership_tier status	PK product_id product_name category price stock
Orders	Order_Items
PK order_id FK customer_id total_amount status order_date	PK item_id FK order_id FK product_id quantity price

Customers 1 : N Orders | Orders 1 : N Order_Items | Products 1 : N Order_Items

PK = Primary Key (khóa chính) **FK** = Foreign Key (khóa ngoại, liên kết bảng khác)

Bảng	Lưu gì	Ví dụ trong sách
Customers	Danh sách khách hàng: họ tên, email, hạng thành viên (Standard / Silver / Gold), trạng thái tài khoản.	C001 = Nguyen Van A, Silver, ACTIVE.
Products	Danh mục sản phẩm: tên, danh mục, giá niêm yết, số lượng tồn kho.	PROD_001 = iPhone 15 Pro Max, giá 30.000.000, tồn 50.
Orders	Mỗi đơn hàng: ai đặt, trạng thái (PENDING / COMPLETED / CANCELLED), ngày đặt, và total_amount — tổng tiền toàn đơn. Giá trị này phải bằng SUM(price × quantity) của tất cả dòng Order_Items cùng order_id.	ORD_001 của C001, tổng 32.000.000, COMPLETED.
Order_Items	Chi tiết từng sản phẩm trong đơn — bảng nối Orders ↔ Products. Mỗi dòng = 1 sản phẩm: mua gì, số lượng, và price (giá 1 đơn vị lúc mua — lưu riêng vì giá sản phẩm có thể thay đổi sau).	ORD_001 có 2 sản phẩm → 2 dòng cùng order_id = ORD_001.

□□ **Lưu ý:** **total_amount** (Orders) và **price** (Order_Items) là hai cột khác nhau. total_amount = tổng cả đơn; price = giá 1 đơn vị. Nếu hai con số này không khớp là có bug — ví dụ: ORD_002 ghi total_amount = 20.000.000 nhưng SUM(price × quantity) trong Order_Items = 31.000.000 → lệch 11.000.000 (Bug 1 trong data mẫu).

1.2 Script tạo Database và bơm dữ liệu mẫu

Thay vì copy-paste thủ công, hãy tải file SQL sẵn có và chạy một lần duy nhất — database **ecommerce_test** cùng toàn bộ dữ liệu mẫu sẽ được tạo tự động.

Tải file: maiqa.com/books/ecommerce_test_setup.sql

Chạy trong MySQL Workbench: File → Open SQL Script → chọn file vừa tải → nhấn **Ctrl+Shift+Enter**.

Chạy bằng CLI: `mysql -u root -p < ecommerce_test_setup.sql`

Script tự xóa và tạo lại data mỗi lần chạy — tiện để reset sau khi thực hành.

1.3 Lỗi đã cài sẵn trong dữ liệu mẫu

Data mẫu chứa đầy đủ lỗi cho cả 50 câu SQL. Dưới đây là tổng hợp theo nhóm để bạn nắm trước khi bắt đầu thực hành.

Trùng lặp & toàn vẹn (Câu 1–8)

Email trùng (C004/C005 · C001/C009), composite key trùng trong Order_Items (item 1 và 7), email trùng không phân biệt hoa/thường, tên sản phẩm trùng hoàn toàn (PROD_002/PROD_005).

NULL & dữ liệu thiếu (Câu 3, 7)

C006: email = NULL. C007: email = chuỗi rỗng. PROD_006: price = NULL. PROD_007: stock = NULL.

Định dạng không hợp lệ (Câu 4, 9)

C008: customer_name có khoảng trắng thừa ở đầu và cuối (' Pham Van D '). C010: membership_tier = 'VIP' — ngoài danh sách Standard/Silver/Gold/Platinum.

Orphan & ràng buộc tham chiếu (Câu 5)

ORD_004: customer_id = 'C999' không tồn tại trong bảng Customers.

Giá trị nghiệp vụ sai (Câu 10, 17, 18)

item_id bỏ trống số 3 — gap trong chuỗi ID liên tục. PROD_003: stock = -5 (tồn kho âm). ORD_002: total_amount = 20.000.000 nhưng Order_Items tổng = 31.000.000 (lệch 11.000.000).

1.4 Dữ liệu mẫu sau khi chạy script

Bảng Customers (10 dòng — dòng đỏ: lỗi trùng email, NULL, khoảng trắng, tier sai)

customer_id	customer_name	email	membership_tier	status
C001	Nguyen Van A	a.nguyen@email.com	Silver	ACTIVE
C002	Tran Van B	b.tran@email.com	Standard	ACTIVE
C003	Le Thi C	c.le@email.com	Gold	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	Standard	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	Standard	ACTIVE
C006	Pham Van X	(NULL)	Standard	ACTIVE
C007	Nguyen Thi Y		Standard	ACTIVE
C008	Pham Van D	d.pham@email.com	Gold	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	Silver	ACTIVE
C010	Khach Test VIP	vip@email.com	VIP	ACTIVE

Bảng Products (7 dòng — dòng đỏ: stock âm, NULL, tên trùng)

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

Bảng Orders (4 dòng — dòng đỏ: total_amount sai, orphan customer)

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24

Bảng Order_Items (6 dòng — dòng đỏ: composite key trùng; item_id=3 bị bỏ qua)

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000

item_id nhảy từ 2 lên 4 (thiếu 3). Item 7 trùng (order_id, product_id) với item 1. ORD_002: tổng items = 31.000.000 nhưng total_amount = 20.000.000 (lệch 11.000.000).

Mục lục

PHẦN 1 · Toàn vẹn và trùng lặp dữ liệu

01. Tìm email bị trùng
02. Tìm trùng theo nhiều cột (composite key)
03. Tìm NULL ở cột bắt buộc
04. Phát hiện khoảng trống thừa và ký tự ẩn
05. Kiểm tra bản ghi mồ côi (foreign key orphan)
06. Tìm trùng không phân biệt hoa/thường
07. Đếm giá trị NULL theo từng cột
08. Tìm bản ghi trùng hoàn toàn (full duplicate)
09. Kiểm tra giá trị ngoài danh sách cho phép (ENUM check)
10. Tìm khoảng trống trong chuỗi ID liên tục

PHẦN 2 · Ràng buộc nghiệp vụ

11. Đối soát tổng tiền đơn hàng với chi tiết items
12. Phát hiện tồn kho âm
13. Phát hiện tồn kho bị NULL
14. Phát hiện giá sản phẩm NULL hoặc bằng 0
15. Tìm đơn hàng không có dòng chi tiết nào
16. Tìm sản phẩm chưa bao giờ được bán
17. Tìm khách hàng chưa có đơn hàng nào
18. Kiểm tra trạng thái đơn hàng ngoài danh sách cho phép
19. Tổng hợp chi tiêu theo từng khách hàng
20. Phát hiện sản phẩm đã bán vượt quá tồn kho

PHẦN 3 · Đối soát và tính toán

21. Tính doanh thu thực theo từng đơn từ Order_Items
22. Xếp hạng sản phẩm bán chạy nhất theo doanh số
23. Kiểm tra giá trị tồn kho (stock × price)
24. So sánh giá bán lịch sử với giá niêm yết hiện tại
25. Tổng doanh thu chỉ tính đơn hàng COMPLETED
26. Phân tích tỷ lệ đơn hàng theo trạng thái
27. Tổng doanh thu theo danh mục sản phẩm
28. Đối soát tồn kho hiện tại với lượng đã bán ra
29. Phát hiện đơn hàng bị tạo trùng (double order)
30. Phát hiện đơn hàng có giá trị bất thường (outlier)

PHẦN 4 · Biên và dữ liệu bất thường

31. Tìm tên khách hàng chứa ký tự bất thường
32. Tìm email không đúng định dạng cơ bản
33. Kiểm tra số lượng sản phẩm bất thường trong Order_Items
34. Kiểm tra ngày đặt hàng bất thường (tương lai hoặc quá xa quá khứ)
35. Tìm tên sản phẩm trùng sau khi chuẩn hóa
36. Kiểm tra status của Customers ngoài danh sách cho phép
37. Kiểm tra giá bán âm hoặc bằng 0 trong Order_Items
38. Phát hiện đơn có tổng tiền nhưng không có sản phẩm nào
39. Phát hiện tổng items vượt quá 1.5 lần total_amount

40. Snapshot chất lượng dữ liệu: đếm dòng bốn bảng

PHẦN 5 · Audit, log và dấu vết

41. Tìm items trong đơn hàng trở tới sản phẩm không tồn tại

42. Tìm sản phẩm chưa bao giờ được đặt mua (dùng NOT EXISTS)

43. Tìm khách hàng chưa bao giờ đặt hàng (dùng NOT EXISTS)

44. Phát hiện item_id bị nhảy — dấu vết của bản ghi bị xóa

45. Phân tích đơn hàng bị hủy — khách nào hay hủy nhất

PHẦN 6 · Truy vấn nâng cao cho QA

46. Dùng ROW_NUMBER() phát hiện item trùng trong cùng một đơn

47. Dùng CTE + RANK() xếp hạng sản phẩm bán chạy

48. Dùng CTE lồng nhau tìm khách chi tiêu trên mức trung bình

49. Tính doanh thu tích lũy theo thời gian với SUM() OVER()

50. Báo cáo tổng hợp: nhiều loại lỗi trong một câu UNION ALL

PHẦN 1 — Toàn vẹn và trùng lặp dữ liệu

Lỗi dữ liệu kinh điển nhất mà QA bắt gặp: bản ghi nhân đôi, ô bắt buộc bị bỏ trống, khóa ngoại trỏ vào nơi không tồn tại. Nhóm câu lệnh này giúp bạn soi nhanh sức khỏe của một bảng trước khi đi sâu kiểm thử nghiệp vụ.

01 Tìm email bị trùng

TÌNH HUỐNG

Hệ thống đăng ký yêu cầu mỗi email chỉ thuộc về một tài khoản. Tuy nhiên do thiếu ràng buộc **UNIQUE** trên cột email, hai bản ghi C004 và C005 đã lọt vào bảng với cùng một địa chỉ. Hậu quả: gửi email nhầm người, đăng nhập nhầm nhằng, chiến dịch marketing tính trùng người dùng.

Bảng Customers — dòng đỏ: email bị trùng (Bug 3):

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT email,
       COUNT(*) AS so_lan
FROM   Customers
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY so_lan DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL bắt đầu từ đây: tải toàn bộ bảng Customers vào vùng xử lý — 10 dòng trong dữ liệu mẫu.
GROUP BY email	Gom nhóm tất cả dòng có cùng giá trị email lại với nhau. NULL và chuỗi rỗng mỗi loại thành một nhóm riêng.

HAVING COUNT(*) > 1	Lọc nhóm: chỉ giữ nhóm có nhiều hơn 1 bản ghi — tức là email xuất hiện từ 2 lần trở lên. HAVING khác WHERE ở chỗ nó lọc SAU khi gom nhóm.
SELECT email, COUNT(*) AS so_lan	Chỉ đến bước này MySQL mới chiều kết quả : lấy email và đặt tên cột đếm là so_lan .
ORDER BY so_lan DESC	Sắp xếp kết quả cuối cùng: email trùng nhiều nhất nổi lên đầu.

Giải thích tổng thể

Kỹ thuật dùng ở đây là **GROUP BY + HAVING COUNT**: gom nhóm theo giá trị cần kiểm tra, rồi lọc những nhóm vi phạm tính duy nhất.

Đây là mẫu truy vấn nền tảng để phát hiện mọi loại trùng lặp trong bất kỳ cột nào.

Kết quả sau khi query (minh họa)

email	so_lan
a.nguyen@email.com	2
trung_email@email.com	2

2 cặp trùng: **trung_email@email.com** (C004 + C005 — Bug 3) và **a.nguyen@email.com** (C001 + C009 — trùng case-insensitive). Cần xử lý cả hai và thêm ràng buộc UNIQUE trước khi đưa lên production.

GÓC SOI LỖI CỦA TESTER

Trùng chính xác chỉ là bề nổi. Câu lệnh này **không** bắt được hai dạng sau:

- (1) **Trùng khác hoa/thường** — 'Test@Mail.com' và 'test@mail.com' bị coi là hai email khác nhau → dùng Câu 6 để phát hiện.
- (2) **Trùng do khoảng trắng thừa** — ' test@mail.com' khác 'test@mail.com' → dùng Câu 4 để phát hiện.

02

Tìm trùng theo nhiều cột (composite key)

TÌNH HUỐNG

Quy tắc nghiệp vụ: mỗi đơn hàng không được chứa cùng một sản phẩm ở hai dòng riêng biệt. Tổ hợp (**order_id, product_id**) phải là duy nhất trong Order_Items. Nếu app bị double-submit hoặc thiếu kiểm tra, một sản phẩm có thể lọt vào hai lần.

Bảng Order_Items — dòng đỏ: cặp (order_id, product_id) xuất hiện lần 2:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000

CÂU LỆNH SQL

```
SELECT order_id,
       product_id,
       COUNT(*) AS so_lan
FROM   Order_Items
GROUP BY order_id, product_id
HAVING COUNT(*) > 1;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items	MySQL tải toàn bộ bảng Order_Items — 6 dòng trong ví dụ.
GROUP BY order_id, product_id	Gom nhóm theo tổ hợp hai cột. Mỗi nhóm đại diện cho một cặp (đơn hàng, sản phẩm) duy nhất.
HAVING COUNT(*) > 1	Lọc nhóm: chỉ giữ những tổ hợp xuất hiện nhiều hơn 1 lần — vi phạm tính duy nhất nghiệp vụ.
SELECT order_id, product_id, COUNT(*) AS so_lan	Chiều kết quả: tên đơn, tên sản phẩm và số lần trùng.

Giải thích tổng thể

Mẫu **GROUP BY + HAVING COUNT** áp dụng cho khóa tổ hợp.

Thay vì GROUP BY một cột như email, ta GROUP BY nhiều cột để đại diện cho quy tắc duy nhất phức tạp hơn.

Thêm bớt cột trong GROUP BY tùy theo quy tắc nghiệp vụ cần kiểm tra.

Kết quả sau khi query (minh họa)

order_id	product_id	so_lan
ORD_001	PROD_001	2

1 dòng: sản phẩm PROD_001 bị thêm hai lần vào ORD_001. Cần xóa dòng trùng và thêm UNIQUE(order_id, product_id) vào bảng.

GÓC SOI LỖI CỦA TESTER

Trước khi xóa dòng trùng, hãy kiểm tra:

- (1) Hai dòng có item_id khác nhau không?
 - (2) Dòng nào được tạo sau — đó mới là dòng lỗi cần xóa.
- Đừng xóa dựa trên giả định — sai item_id sẽ phá vỡ các bảng liên kết.

03 Tìm NULL ở cột bắt buộc

TÌNH HUỐNG

Cột email được quy định là bắt buộc trên giao diện đăng ký, nhưng luồng import dữ liệu cũ hoặc API nội bộ lại không kiểm tra. Kết quả: một số tài khoản không có email — không thể gửi thông báo, không thể reset mật khẩu, chiến dịch CRM bị lỗi.

Bảng Customers — dòng đỏ: email NULL hoặc chuỗi rỗng:

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE

customer_id	customer_name	email	status
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A. NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name,
       email
FROM   Customers
WHERE  email IS NULL
       OR TRIM(email) = '';
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
WHERE email IS NULL OR TRIM(email) = ''	Lọc hai kiểu rỗng: IS NULL bắt giá trị chưa có, TRIM(email) = '' bắt chuỗi rỗng hoặc chỉ có khoảng trắng. Thiếu một trong hai điều kiện sẽ bỏ sót một kiểu lỗi.
SELECT customer_id, customer_name, email	Chiếu ra các cột để QA xác minh và báo cáo số lượng bị ảnh hưởng.

Giải thích tổng thể

Có **hai kiểu rỗng** hoàn toàn khác nhau trong SQL:

- (1) **NULL** — chưa có giá trị, hệ thống không biết email là gì.
- (2) **Chuỗi rỗng ""** — có giá trị nhưng là rỗng, biết email nhưng không có nội dung.

Điều kiện **email != ""** sẽ **KHÔNG** lọc được NULL vì mọi phép so sánh với NULL đều trả về UNKNOWN.

Ket qua sau khi query (minh hoa)

customer_id	customer_name	email
C006	Pham Van X	(NULL)
C007	Nguyen Thi Y	

Số dòng trả về = quy mô dữ liệu thiếu. Dùng con số này để viết defect report mức độ ảnh hưởng.

GÓC SOI LỖI CỦA TESTER

Đừng chỉ kiểm tra **IS NULL** — đó chỉ là một nửa bức tranh.

(1) **Chuỗi rỗng**: hệ thống cũ thường lưu "" thay cho NULL → cần thêm điều kiện **TRIM(email) = ""**.

(2) **Khoảng trắng thừa**: ' test@email.com' vẫn được lưu nhưng sau TRIM mới lộ ra là chuỗi rỗng → kết hợp cả hai điều kiện vào một câu lệnh.

04

Phát hiện khoảng trắng thừa và ký tự ẩn

TÌNH HUỐNG

Người dùng copy-paste tên từ Excel hoặc nhập trên điện thoại dễ kéo theo dấu cách thừa ở đầu/cuối. Kết quả: 'Tran Van B' và ' Tran Van B ' bị coi là hai người khác nhau — trùng lặp logic nhưng không bị bắt bởi kiểm tra UNIQUE thông thường.

Bảng Customers — dòng đỏ: tên có khoảng trắng thừa:

customer_id	customer_name	status
C001	Nguyen Van A	ACTIVE
C002	Tran Van B	ACTIVE
C003	Le Thi C	ACTIVE
C004	Khach Hang Ao Bug	ACTIVE
C005	Khach Hang Trung	ACTIVE
C006	Pham Van X	ACTIVE
C007	Nguyen Thi Y	ACTIVE
C008	Pham Van D	ACTIVE
C009	Nguyen Van A (2)	ACTIVE
C010	Khach Test VIP	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name,
       CHAR_LENGTH(customer_name)      AS do_dai_goc,
       CHAR_LENGTH(TRIM(customer_name)) AS do_dai_sau_trim
FROM   Customers
WHERE  customer_name != TRIM(customer_name);
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
WHERE customer_name != TRIM(customer_name)	Lọc dòng mà độ dài gốc khác độ dài sau khi cắt trắng — tức là tồn tại khoảng trắng thừa ở đầu hoặc cuối.
SELECT ..., CHAR_LENGTH(customer_name), CHAR_LENGTH(TRIM(...))	Hiển thị độ dài gốc và sau trim để thấy chênh lệch bao nhiêu ký tự .

Giải thích tổng thể

TRIM chỉ cắt khoảng trắng ở hai đầu, không xóa khoảng trắng ở giữa tên.
So sánh CHAR_LENGTH trước và sau TRIM để đo chính xác có bao nhiêu ký tự thừa.
Nếu cần bắt cả tab (\\t) hay xuống dòng (\\n), dùng thêm REPLACE hoặc REGEXP.

Ket qua sau khi query (minh hoa)

customer_id	customer_name	do_dai_goc	do_dai_sau_trim
C008	Pham Van D	14	10

Mỗi dòng trả về cần được chuẩn hóa bằng UPDATE ... SET customer_name = TRIM(customer_name).

GÓC SOI LỖI CỦA TESTER

TRIM trong MySQL chỉ cắt dấu cách thường (ASCII 32).

Nếu dữ liệu nhập từ Excel, có thể tồn tại ký tự **non-breaking space** (u00a0) — TRIM sẽ không bắt được.

Xử lý bổ sung bằng: **REPLACE(customer_name, CHAR(160), ")** trước khi so sánh hoặc chuẩn hóa.

05

Kiểm tra bản ghi mồ côi (foreign key orphan)

TÌNH HUỐNG

Bảng Orders có cột customer_id liên kết sang Customers. Nếu FK constraint bị tắt hoặc dữ liệu được import thủ công, có thể xuất hiện đơn hàng với customer_id không tồn tại — orphan record. Báo cáo doanh thu sẽ sai vì không join được thông tin khách.

Bảng Orders — dòng đỏ: customer_id không có trong Customers:

order_id	customer_id	total_amount	status
ORD_001	C001	32.000.000	COMPLETED
ORD_002	C002	20.000.000	COMPLETED
ORD_003	C003	8.000.000	CANCELLED
ORD_004	C999	5.000.000	PENDING

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.customer_id
FROM   Orders o
       LEFT JOIN Customers c ON o.customer_id = c.customer_id
WHERE  c.customer_id IS NULL;
```

FROM Orders o LEFT JOIN Customers c ON o.customer_id = c.customer_id	LEFT JOIN giữ lại TẤT CẢ dòng trong Orders — kể cả dòng không khớp với bất kỳ dòng nào trong Customers. Với INNER JOIN , dòng orphan sẽ biến mất và không phát hiện được.
WHERE c.customer_id IS NULL	Sau LEFT JOIN , những dòng Orders không khớp sẽ có c.customer_id = NULL. Lọc đúng các dòng đó.
SELECT o.order_id, o.customer_id	Chiếu ra order_id và customer_id để xác định đơn hàng nào bị mồ côi.

Giải thích tổng thể

Kỹ thuật **LEFT JOIN + WHERE IS NULL** là cách chuẩn để tìm orphan record.

Nguyên lý: **LEFT JOIN** giữ nguyên tất cả dòng bên trái. Những dòng không có cặp bên phải sẽ có giá trị **NULL** ở mọi cột của bảng phải.

Ta lọc đúng điều kiện đó để xác định bản ghi mồ côi.

Ket qua sau khi query (minh họa)

order_id	customer_id
ORD_004	C999

ORD_004 không có chủ. Cần xác minh: customer C999 có thực tồn tại nhưng bị xóa nhầm, hay đây là đơn hàng được tạo bởi bug?

GÓC SOI LỖI CỦA TESTER

Câu lệnh này kiểm tra chiều **Orders** → **Customers** (đơn hàng không có khách hàng).

Cần kiểm tra thêm các chiều khác:

(1) **Order_Items** → **Products**: item có product_id không tồn tại (xem Câu 41).

(2) **Order_Items** → **Orders**: item thuộc order_id không tồn tại.

Cả hai đều là orphan nhưng ở tầng bảng khác nhau.

06 Tìm trùng không phân biệt hoa/thường**TÌNH HUỐNG**

Database lưu email theo kiểu người dùng nhập — 'Test@Mail.com' và 'test@mail.com' là cùng một địa chỉ nhưng MySQL mặc định phân biệt hoa/thường trong một số collation. Kiểm tra UNIQUE trên cột gốc sẽ bỏ qua dạng trùng này.

Bảng Customers — dòng đỏ: email trùng khi so sánh không phân biệt hoa/thường:

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A. NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT LOWER(email) AS email_chuan,
       COUNT(*)      AS so_lan
FROM   Customers
GROUP BY LOWER(email)
HAVING COUNT(*) > 1
ORDER BY so_lan DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
GROUP BY LOWER(email)	LOWER(email) chuyển về chữ thường trước khi gom nhóm. Nhờ vậy 'A. NGUYEN@EMAIL.COM' và 'a.nguyen@email.com' rơi vào cùng một nhóm.
HAVING COUNT(*) > 1	Chỉ giữ nhóm có nhiều hơn 1 bản ghi — email trùng sau khi chuẩn hóa.

SELECT LOWER(email) AS email_chuan, COUNT(*) AS so_lan	Chiều email đã chuẩn hóa và số lần trùng.
ORDER BY so_lan DESC	Email trùng nhiều nhất hiện lên đầu.

Giải thích tổng thể

Bọc cột trong hàm **LOWER()** trước GROUP BY là kỹ thuật chuẩn hóa trước khi gom nhóm.
Áp dụng tương tự với **UPPER()**, **TRIM()**, hay kết hợp cả hai tùy theo loại dữ liệu cần kiểm tra.

Kết quả sau khi query (minh họa)

email_chuan	so_lan
a.nguyen@email.com	2
trung_email@email.com	2

2 cặp trùng sau khi chuẩn hóa hoa/thường: **a.nguyen@email.com** (C001 + C009) và **trung_email@email.com** (C004 + C005). Câu 1 đã bắt được trung_email — câu này bắt thêm cặp chỉ trùng về case.

GÓC SOI LỖI CỦA TESTER

Câu lệnh này chỉ cần thiết khi collation của cột là **case-sensitive** (vd: utf8mb4_bin).
Nếu bảng dùng **utf8mb4_unicode_ci** (case-insensitive), MySQL tự động so sánh không phân biệt hoa/thường — Câu 1 đã đủ, câu này thừa.
Kiểm tra collation trước khi quyết định dùng: **SHOW FULL COLUMNS FROM Customers;**

07 Đếm giá trị NULL theo từng cột

TÌNH HUỐNG

Thay vì kiểm tra NULL từng cột riêng lẻ, câu lệnh này cho ra một bảng tổng hợp — mỗi cột một ô — để QA thấy ngay bức tranh toàn cảnh: cột nào bị thiếu nhiều nhất, ưu tiên xử lý cột nào trước.

Bảng Products — dòng đỏ: có giá trị NULL trong cột số:

product_id	product_name	price	stock
PROD_001	iPhone 15 Pro Max	30.000.000	50
PROD_002	Bàn phím cơ Logitech	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	8.000.000	-5
PROD_004	Sạc du phong Anker	1.000.000	20
PROD_005	Bàn phím cơ Logitech	2.000.000	30
PROD_006	Loa Bluetooth JBL	(NULL)	10
PROD_007	Chuột gaming Razer	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT
  SUM(CASE WHEN product_name IS NULL THEN 1 ELSE 0 END) AS null_ten,
  SUM(CASE WHEN price        IS NULL THEN 1 ELSE 0 END) AS null_gia,
  SUM(CASE WHEN stock        IS NULL THEN 1 ELSE 0 END) AS null_ton
FROM Products;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products .
SUM(CASE WHEN ... IS NULL THEN 1 ELSE 0 END)	Với mỗi dòng, CASE WHEN trả về 1 nếu cột là NULL, 0 nếu không. SUM cộng lại — kết quả là tổng số NULL của cột đó. Áp dụng lặp lại cho từng cột cần kiểm tra.
SELECT null_ten, null_gia, null_ton	Kết quả là 1 dòng duy nhất chứa số NULL của mỗi cột — dạng báo cáo nhanh cho QA.

Giải thích tổng thể

Kỹ thuật **CASE WHEN + SUM** là cách pivot đơn giản: chuyển điều kiện thành số rồi tổng hợp. Không cần GROUP BY vì toàn bộ bảng là một nhóm duy nhất — kết quả trả về chỉ 1 dòng. Mở rộng bằng cách thêm cột vào SELECT để kiểm tra nhiều trường trong một lần chạy.

Ket qua sau khi query (minh hoa)

null_ten	null_gia	null_ton
0	1	1

Cột price và stock đều có 1 NULL — cần điều tra nguồn dữ liệu và quyết định giá trị mặc định.

GÓC SOI LỖI CỦA TESTER

Mở rộng câu lệnh sang bảng **Customers** để tạo báo cáo chất lượng dữ liệu tổng thể:

(1) Đếm NULL theo từng cột: **null_email, null_tier, null_status**.

(2) Chạy ngay đầu sprint — phát hiện sớm giúp team thống nhất giá trị mặc định trước khi viết test case.

08

Tìm bản ghi trùng hoàn toàn (full duplicate)

TÌNH HUỐNG

Khác với trùng ID, full duplicate là hai dòng có toàn bộ giá nghiệp vụ giống nhau nhưng ID tự sinh khác nhau — thường xảy ra khi người dùng bấm nút 'Lưu' hai lần hoặc import dữ liệu không kiểm tra trùng trước.

Bảng Products — dòng đỏ: cùng tên và giá với dòng khác:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT product_name,  
       price,  
       COUNT(*) AS so_lan  
FROM   Products  
GROUP BY product_name, price  
HAVING COUNT(*) > 1;
```


PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products .
GROUP BY product_name, price	Gom nhóm theo tổ hợp các cột nghiệp vụ — ở đây là tên sản phẩm và giá. Thêm category vào GROUP BY nếu muốn kiểm tra chặt hơn.
HAVING COUNT(*) > 1	Lọc nhóm có nhiều hơn 1 dòng — tức là cùng tên và giá xuất hiện lặp lại.
SELECT product_name, price, COUNT(*) AS so_lan	Chiếu tên, giá và số lần trùng để xác định bản ghi cần xóa.

Giải thích tổng thể

Định nghĩa 'trùng hoàn toàn' phụ thuộc vào nghiệp vụ: có thể trùng theo 2, 3 hoặc toàn bộ cột.
Hãy thêm bớt cột trong GROUP BY tùy theo ngữ cảnh — chỉ đưa vào những cột đại diện cho giá trị nghiệp vụ, không phải ID tự sinh.

Ket qua sau khi query (minh hoa)

product_name	price	so_lan
Ban phim co Logitech	2.000.000	2

Cần xác định PROD_002 hay PROD_005 là bản gốc trước khi xóa. Kiểm tra xem bản nào có Order_Items liên kết.

GÓC SOI LỖI CỦA TESTER

Trước khi xóa, hãy xem đầy đủ cả hai dòng để quyết định đúng:

SELECT * FROM Products

WHERE product_name = 'Ban phim co Logitech' AND price = 2000000;

Kiểm tra thêm: dòng nào đang có **Order_Items** liên kết — đó là bản gốc, dòng còn lại mới là bản trùng cần xóa.

09

Kiểm tra giá trị ngoài danh sách cho phép (ENUM check)

TÌNH HUỐNG

Cột membership_tier chỉ được nhận một trong bốn giá trị: Standard, Silver, Gold, Platinum. Nếu tầng ứng dụng không validate, hoặc dữ liệu được nhập thẳng vào DB qua script, các giá trị lạ sẽ lọt vào và làm vỡ logic ưu đãi.

Bảng Customers — dòng đỏ: membership_tier ngoài danh sách hợp lệ:

customer_id	customer_name	membership_tier	status
C001	Nguyen Van A	Silver	ACTIVE
C002	Tran Van B	Standard	ACTIVE
C003	Le Thi C	Gold	ACTIVE
C004	Khach Hang Ao Bug	Standard	ACTIVE
C005	Khach Hang Trung	Standard	ACTIVE
C006	Pham Van X	Standard	ACTIVE
C007	Nguyen Thi Y	Standard	ACTIVE

customer_id	customer_name	membership_tier	status
C008	Pham Van D	Gold	ACTIVE
C009	Nguyen Van A (2)	Silver	ACTIVE
C010	Khach Test VIP	VIP	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name,
       membership_tier
FROM   Customers
WHERE  membership_tier NOT IN ('Standard','Silver','Gold','Platinum');
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
WHERE membership_tier NOT IN ('Standard','Silver','Gold','Platinum')	NOT IN lọc tất cả dòng có giá trị không nằm trong danh sách. Lưu ý: NULL sẽ KHÔNG khớp NOT IN — cần thêm OR membership_tier IS NULL nếu muốn bắt cả NULL.
SELECT customer_id, customer_name, membership_tier	Chiếu đủ thông tin để QA xác minh và cập nhật về giá trị đúng.

Giải thích tổng thể

NOT IN + danh sách tường minh là cách nhanh nhất để kiểm tra ENUM không được enforce ở tầng DB. Áp dụng tương tự cho cột status của Orders: NOT IN ('COMPLETED','PENDING','CANCELLED','PROCESSING').

Kết quả sau khi query (minh họa)

customer_id	customer_name	membership_tier
C010	Khach Test VIP	VIP

1 bản ghi có tier không hợp lệ. Cần cập nhật về giá trị đúng và bổ sung CHECK constraint hoặc ENUM type trong DB.

GÓC SOI LỖI CỦA TESTER

Cạm bẫy với **NOT IN**: nếu danh sách chứa NULL, toàn bộ điều kiện trả về UNKNOWN. Ví dụ: **WHERE tier NOT IN ('Standard', NULL)** — không trả về dòng nào, kể cả dòng có tier = 'VIP'. Quy tắc: **luôn đảm bảo danh sách NOT IN không chứa NULL** — hoặc dùng NOT EXISTS thay thế để an toàn hơn.

10 Tìm khoảng trống trong chuỗi ID liên tục

TÌNH HUỐNG

Sau khi xóa hoặc rollback giao dịch, chuỗi item_id AUTO_INCREMENT có thể bị gián đoạn. Đây không nhất thiết là bug — nhưng nếu nghiệp vụ yêu cầu số phiếu liên tục (biên lai, hóa đơn), khoảng trống là dấu hiệu cần kiểm tra.

Bảng Order_Items — dòng đỏ: item_id=4 bị nhảy cách (thiếu item_id=3):

item_id	order_id	product_id	quantity
1	ORD_001	PROD_001	1
2	ORD_001	PROD_002	1
4	ORD_002	PROD_001	1
5	ORD_002	PROD_004	1
6	ORD_003	PROD_003	1
7	ORD_001	PROD_001	1

CÂU LỆNH SQL

```
SELECT t1.item_id + 1 AS id_bi_thieu
FROM   Order_Items t1
WHERE  NOT EXISTS (
    SELECT 1
    FROM   Order_Items t2
    WHERE  t2.item_id = t1.item_id + 1
)
AND t1.item_id < (SELECT MAX(item_id) FROM Order_Items);
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items t1	Quét từng dòng trong bảng — mỗi dòng đại diện cho một ID đang tồn tại.
WHERE NOT EXISTS (SELECT 1 FROM Order_Items t2 WHERE t2.item_id = t1.item_id + 1)	NOT EXISTS: với mỗi item_id, kiểm tra xem item_id+1 có tồn tại không. Nếu không — đó là khoảng trống.
AND t1.item_id < (SELECT MAX(item_id) FROM Order_Items)	Chỉ kiểm tra đến ID lớn nhất — bỏ qua item_id cuối (không có 'lỗi' nếu chưa có tiếp theo).
SELECT t1.item_id + 1 AS id_bi_thieu	Chiều ID bị thiếu = item_id hiện tại + 1.

Giải thích tổng thể

Kỹ thuật **self-join NOT EXISTS**: so sánh bảng với chính nó (alias t1 và t2) để tìm ID nào mà người kế tiếp không tồn tại.
Phù hợp với bảng nhỏ — cách viết thay thế dùng self-join cho kết quả tương đương với cú pháp khác (xem Câu 44).

Ket qua sau khi query (minh họa)

id_bi_thieu
3

item_id = 3 bị thiếu. Kiểm tra log: có giao dịch nào bị rollback gần thời điểm đó không?

GÓC SOI LỖI CỦA TESTER

AUTO_INCREMENT không đảm bảo liên tục — đây là đặc điểm thiết kế của MySQL, không phải bug. Gặp xảy ra bình thường khi: INSERT bị rollback, bản ghi bị xóa, hoặc server restart.

Nếu nghiệp vụ **bắt buộc** số phiếu liên tục (vd: hóa đơn thuế), cần implement sequence riêng thay vì dùng AUTO_INCREMENT.

PHẦN 2 — Ràng buộc nghiệp vụ

Mỗi hệ thống đều có những quy tắc bất thành văn: tổng tiền không thể âm, tồn kho không thể dưới không, trạng thái phải nằm trong danh sách cho phép. Khi tăng ứng dụng quên kiểm tra, dữ liệu sai sẽ lặn lẽ trôi vào database.

11 Đối soát tổng tiền đơn hàng với chi tiết items

TÌNH HUỐNG

Hệ thống lưu **total_amount** trong Orders nhưng không tự đối soát với tổng tính từ Order_Items. Khi có dữ liệu nhân đôi hoặc bug logic, hai con số lặn lẽ lệch nhau mà không có cảnh báo nào.

Bảng Orders — dòng đồ: total_amount lệch với tổng Order_Items:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.total_amount           AS ghi_trong_don,
       SUM(i.quantity * i.price) AS tinh_tu_items,
       o.total_amount
       - SUM(i.quantity * i.price) AS chenh_lech
FROM   Orders o
JOIN   Order_Items i ON o.order_id = i.order_id
GROUP BY o.order_id, o.total_amount
HAVING SUM(i.quantity * i.price) != o.total_amount;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o JOIN Order_Items i ON o.order_id = i.order_id	INNER JOIN ghép mỗi đơn hàng với tất cả items của nó. Đơn không có item sẽ bị bỏ qua — dùng LEFT JOIN + Câu 15 để bắt thêm.
GROUP BY o.order_id, o.total_amount	Gom toàn bộ items của cùng một đơn thành một nhóm. total_amount cần có trong GROUP BY vì được dùng trong HAVING.
HAVING SUM(i.quantity * i.price) != o.total_amount	HAVING lọc sau khi đã tính aggregate — chỉ giữ đơn có tổng items khác với total_amount đã ghi.
SELECT ..., chenh_lech	Hiển thị cả ba con số: ghi bao nhiêu, tính ra bao nhiêu, chênh lệch bao nhiêu — đủ thông tin để QA viết defect report.

Giải thích tổng thể

Kỹ thuật **JOIN + GROUP BY + HAVING** để đối soát header-detail: total_amount trong Orders phải bằng SUM(quantity × price) từ Order_Items.

Kết quả phát hiện **hai nguyên nhân khác nhau**:

(1) ORD_001 lệch vì item bị nhân đôi (item 1 và item 7 cùng order + product).

(2) ORD_002 lệch vì total_amount bị ghi sai thủ công (Bug 1).

Cùng triệu chứng nhưng cách xử lý khác nhau — cần tra log để phân biệt.

Ket qua sau khi query (minh hoa)

order_id	ghi_trong_don	ting_tu_items	chenh_lech
ORD_001	32.000.000	62.000.000	-30.000.000
ORD_002	20.000.000	31.000.000	-11.000.000

2 đơn lệch: ORD_001 do item trùng (item 7 nhân đôi item 1 → tổng items = 62M); ORD_002 do total_amount ghi sai (Bug 1). Cùng triệu chứng — khác nguyên nhân.

GÓC SOI LỖI CỦA TESTER

INNER JOIN làm ẩn các đơn không có item — những đơn này có thể có total_amount > 0 nhưng không bao giờ xuất hiện trong kết quả câu này.

Kết hợp với **Câu 15** để kiểm tra toàn diện:

(1) Câu 11: đơn có item nhưng tổng lệch → bug tính toán hoặc dữ liệu nhân đôi.

(2) Câu 15: đơn không có item nào → trường hợp nghiêm trọng hơn.

12 Phát hiện tồn kho âm

TÌNH HUỐNG

Nhiệm vụ yêu cầu tồn kho không thể xuống dưới 0. Nhưng nếu thiếu validation hoặc có race condition khi nhiều người cùng đặt hàng, cột **stock** vẫn có thể mang giá trị âm mà không bị hệ thống chặn.

Bảng Products — dòng đỏ: stock âm (Bug 2):

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	NULL	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	NULL

CÂU LỆNH SQL

```
SELECT product_id,
       product_name,
       stock
FROM   Products
WHERE  stock < 0;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products .
WHERE stock < 0	Lọc sản phẩm có tồn kho âm — vi phạm ràng buộc nghiệp vụ. Đổi thành stock <= 0 nếu muốn bắt cả trường hợp hết hàng.
SELECT product_id, product_name, stock	Chiếu đủ thông tin để QA xác định sản phẩm và mức độ lệch.

Giải thích tổng thể

Câu lệnh đơn giản nhưng **cực kỳ quan trọng** trong kiểm thử e-commerce.

Tồn kho âm thường xảy ra ở hai tình huống:

- (1) **Thiếu CHECK constraint**: DB không có quy tắc chặn giá trị âm. Khi code chạy UPDATE stock = stock - 1 lúc stock đang = 0, DB chấp nhận ngay — kết quả stock = -1 mà không báo lỗi gì.
- (2) **Race condition**: Hai người cùng đặt hàng lúc còn 1 sản phẩm. Cả hai cùng đọc stock = 1 trong tích tắc, cùng kiểm tra 'còn hàng', rồi cùng trừ 1 — kết quả cuối là stock = -1. Xảy ra vì hai thao tác chạy song song, không chờ nhau.

Phát hiện sớm giúp team thêm CHECK constraint hoặc khóa giao dịch kịp thời.

Ket qua sau khi query (minh họa)

product_id	product_name	stock
PROD_003	Tai nghe Sony WH-1000XM5	-5

PROD_003 tồn kho = -5. Cần điều tra: lỗi dữ liệu hay hệ thống đã cho phép bán quá số lượng?

GÓC SOI LỖI CỦA TESTER

Thêm **CHECK constraint** vào DB để ngăn từ đầu:

ALTER TABLE Products ADD CONSTRAINT chk_stock CHECK (stock >= 0);

Constraint này chỉ chặn INSERT/UPDATE mới — dữ liệu âm đã có vẫn giữ nguyên.

Kết hợp với **Câu 13** (stock IS NULL) để kiểm tra đầy đủ cột stock:

- (1) Câu 12: stock âm — bán quá số lượng.
- (2) Câu 13: stock NULL — dữ liệu chưa được điền.

13 Phát hiện tồn kho bị NULL

TÌNH HUỐNG

Cột stock bị NULL khác với stock = 0 (hết hàng). NULL nghĩa là **không có thông tin** — hệ thống không biết còn hay hết. Nếu không phân biệt, trang sản phẩm có thể hiển thị sai trạng thái 'còn hàng' cho sản phẩm thực ra chưa được nhập kho.

Bảng Products — dòng đỏ: stock = NULL (dữ liệu thiếu):

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	NULL	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	NULL

CÂU LỆNH SQL

```
SELECT product_id,
       product_name,
       stock
FROM   Products
WHERE  stock IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products .
WHERE stock IS NULL	IS NULL là toán tử duy nhất để so sánh với NULL. Không thể dùng = NULL vì NULL = NULL trả về UNKNOWN, không phải TRUE.
SELECT product_id, product_name, stock	Chiếu ra để QA xác định và bổ sung giá trị còn thiếu.

Giải thích tổng thể

NULL trong SQL không phải số không, không phải chuỗi rỗng — là giá trị **không xác định**. Mọi phép so sánh với NULL đều trả về UNKNOWN: **stock = NULL → UNKNOWN**, **stock != NULL → UNKNOWN**. Do đó phải dùng **IS NULL** hoặc **IS NOT NULL** — không thể dùng = hay !=. Đây là lỗi lập trình phổ biến nhất khi mới làm việc với SQL.

Kết quả sau khi query (minh họa)

product_id	product_name	stock
PROD_007	Chuot gaming Razer	NULL

PROD_007 chưa có thông tin tồn kho. Cần điền giá trị thực hoặc quyết định giá trị mặc định trước khi cho phép bán.

GÓC SOI LỖI CỦA TESTER

Kết hợp Câu 12 và Câu 13 thành một câu để có báo cáo đầy đủ:
WHERE stock IS NULL OR stock < 0
Kết quả: PROD_003 (stock = -5) và PROD_007 (stock = NULL) — hai loại lỗi, hai cách xử lý:
(1) stock âm → điều tra race condition hoặc logic trừ kho.
(2) stock NULL → bổ sung dữ liệu hoặc thêm DEFAULT 0 vào schema.

14 Phát hiện giá sản phẩm NULL hoặc bằng 0

TÌNH HUỐNG

Sản phẩm có **price = NULL** hoặc **price = 0** là dữ liệu chưa hoàn chỉnh. Nếu trang web tính tổng giỏ hàng bằng SUM(price × quantity), NULL sẽ làm toàn bộ tổng trả về NULL; price = 0 cho phép mua miễn phí ngoài ý muốn.

Bảng Products — dòng đỏ: price = NULL (chưa nhập giá):

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5

product_id	product_name	category	price	stock
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	NULL	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	NULL

CÂU LỆNH SQL

```
SELECT product_id,
       product_name,
       price
FROM   Products
WHERE  price IS NULL
       OR price <= 0;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products .
WHERE price IS NULL OR price <= 0	IS NULL bắt giá chưa nhập; <= 0 bắt giá âm hoặc bằng 0. Dùng OR để kiểm tra cả hai trường hợp trong một câu lệnh.
SELECT product_id, product_name, price	Chiếu thông tin để QA xác định sản phẩm cần bổ sung hoặc sửa giá.

Giải thích tổng thể

Giá là trường nghiệp vụ quan trọng nhất trong e-commerce.

Câu lệnh bắt hai loại lỗi:

- (1) **NULL** — chưa nhập giá, thường xảy ra khi sản phẩm mới được thêm vào nhưng chưa định giá.
- (2) **<= 0** — giá không hợp lệ: âm hoặc bằng 0 ngoài ý muốn.

Trong data mẫu, PROD_006 chưa có giá — nếu người dùng thêm vào giỏ hàng, tổng đơn sẽ trả về NULL.

Ket qua sau khi query (minh hoa)

product_id	product_name	price
PROD_006	Loa Bluetooth JBL	NULL

PROD_006 chưa có giá. Cần bổ sung trước khi cho phép bán — hoặc ẩn sản phẩm này khỏi giao diện người dùng.

GÓC SOI LỖI CỦA TESTER

Để ngăn dữ liệu lỗi ngay từ đầu, schema nên có hai ràng buộc:

- (1) **NOT NULL** — bắt buộc nhập giá, không cho để trống.
- (2) **CHECK (price > 0)** — giá phải lớn hơn 0, không cho nhập âm hoặc bằng 0.

Nếu hệ thống có sản phẩm miễn phí hợp lệ, đổi thành **CHECK (price >= 0)** để cho phép price = 0 có chủ đích.

Tránh đặt **DEFAULT 0** cho cột price: khi tạo sản phẩm mới mà quên nhập giá, DB sẽ tự điền 0 thay vì báo lỗi — sản phẩm lặng lẽ lên sàn với giá miễn phí.

15

Tìm đơn hàng không có dòng chi tiết nào

TÌNH HUỐNG

Mỗi đơn hàng phải có ít nhất một sản phẩm. Đơn rỗng — tồn tại trong Orders nhưng không có dòng nào trong Order_Items — là dấu hiệu bug luồng xử lý: đơn được tạo nhưng bước thêm sản phẩm bị lỗi hoặc bị bỏ qua.

Bảng Orders — dòng đỏ: ORD_004 không có item nào trong Order_Items:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.customer_id,
       o.total_amount,
       o.status
FROM   Orders o
LEFT JOIN Order_Items i
      ON o.order_id = i.order_id
WHERE  i.order_id IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o LEFT JOIN Order_Items i ON o.order_id = i.order_id	LEFT JOIN giữ TẤT CẢ đơn hàng, kể cả đơn không có dòng nào trong Order_Items. Với INNER JOIN, đơn rỗng sẽ biến mất.
WHERE i.order_id IS NULL	Sau LEFT JOIN, đơn không khớp có toàn bộ cột của Order_Items = NULL. Lọc những dòng đó là cách nhận diện đơn rỗng — đây là mẫu anti-join.
SELECT o.order_id, o.customer_id, o.total_amount, o.status	Hiển thị đủ thông tin để QA điều tra: ai đặt, số tiền bao nhiêu, trạng thái gì.

Giải thích tổng thể

Kỹ thuật **LEFT JOIN + WHERE IS NULL** (anti-join) là cách chuẩn để tìm bản ghi không có con. Nguyên lý: sau LEFT JOIN, mọi cột từ bảng bên phải sẽ là NULL với những dòng không khớp. Trong data mẫu, ORD_004 (khách C999 không tồn tại) cũng không có item nào — đây là đơn có **hai lỗi chồng nhau**: orphan customer và rỗng items.

Ket qua sau khi query (minh họa)

order_id	customer_id	total_amount	status
ORD_004	C999	5.000.000	PENDING

ORD_004 không có item nào và customer_id C999 không tồn tại. Hai lỗi chồng nhau — cần điều tra lịch sử tạo đơn.

GÓC SOI LỖI CỦA TESTER

Anti-join LEFT JOIN + WHERE IS NULL là pattern dùng lại ở nhiều câu:
(1) Câu 5: Orders không có Customers — đơn hàng mở cối.
(2) Câu 15: Orders không có Order_Items — đơn rỗng.
(3) Câu 16: Products không có Order_Items — sản phẩm chưa bán.
Ba câu cùng kỹ thuật nhưng kiểm tra ở ba tầng quan hệ khác nhau.

16

Tìm sản phẩm chưa bao giờ được bán

TÌNH HUỐNG

Sản phẩm có trong danh mục nhưng không xuất hiện trong bất kỳ đơn hàng nào. Có thể là sản phẩm mới chưa ra mắt, sản phẩm bị ẩn nhưng dữ liệu chưa dọn, hoặc sản phẩm bị lỗi khiến không ai thêm được vào giỏ hàng.

Bảng Products — dòng đỏ: chưa có trong Order_Items:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	NULL	10
PROD_007	Chuat gaming Razer	Phu kien	1.500.000	NULL

CÂU LỆNH SQL

```
SELECT p.product_id,
       p.product_name,
       p.price,
       p.stock
FROM   Products p
LEFT JOIN Order_Items oi
      ON p.product_id = oi.product_id
WHERE  oi.product_id IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products p LEFT JOIN Order_Items oi ON p.product_id = oi.product_id	LEFT JOIN giữ TẤT CẢ sản phẩm, kể cả sản phẩm không có dòng nào trong Order_Items.
WHERE oi.product_id IS NULL	Sau LEFT JOIN, sản phẩm không khớp có oi.product_id = NULL. Đây chính xác là sản phẩm chưa bao giờ được bán.
SELECT p.product_id, p.product_name, p.price, p.stock	Chiếu thêm price và stock để QA đánh giá: chưa bán vì lỗi, chưa có giá, hay chưa có hàng?

Giải thích tổng thể

Cùng kỹ thuật anti-join với Câu 5 và Câu 15, áp dụng cho chiều Products → Order_Items.
Trong data mẫu, 3 sản phẩm chưa bán — mỗi cái có lý do khác nhau:

(1) PROD_005: trùng tên PROD_002 — sản phẩm bị nhân đôi.

(2) PROD_006: price = NULL — chưa định giá.

(3) PROD_007: stock = NULL — chưa nhập kho.

Kết hợp với Câu 13 và Câu 14 để phân tích từng trường hợp.

Kết quả sau khi query (minh họa)

product_id	product_name	price	stock
PROD_005	Bàn phím cơ Logitech	2.000.000	30
PROD_006	Loa Bluetooth JBL	NULL	10
PROD_007	Chuột gaming Razer	1.500.000	NULL

3 sản phẩm chưa bán: PROD_005 trùng tên PROD_002, PROD_006 chưa có giá, PROD_007 chưa có tồn kho.

GÓC SOI LỖI CỦA TESTER

Sản phẩm chưa bán không nhất thiết là lỗi — có thể là hàng mới chưa ra mắt.

Để phân biệt, kết hợp thêm điều kiện từ Câu 13 và Câu 14:

(1) **PROD_005**: trùng với PROD_002 → xác nhận bằng Câu 8 rồi xóa bản thừa.

(2) **PROD_006**: price = NULL → cần nhập giá (Câu 14).

(3) **PROD_007**: stock = NULL → cần nhập tồn kho (Câu 13).

17 Tìm khách hàng chưa có đơn hàng nào

TÌNH HUỐNG

Khách hàng đã đăng ký tài khoản nhưng chưa đặt đơn hàng nào. Có thể là dữ liệu test, tài khoản tạo nhầm, hoặc khách thật nhưng gặp lỗi khi checkout. Nhận diện nhóm này giúp làm sạch dữ liệu trước khi kiểm thử tính năng phân tích người dùng.

Bảng Customers — dòng đỏ: chưa có đơn hàng nào trong Orders:

customer_id	customer_name	membership_tier	status
C001	Nguyen Van A	Silver	ACTIVE
C002	Tran Van B	Standard	ACTIVE
C003	Le Thi C	Gold	ACTIVE
C004	Khách Hàng Ao Bug	Standard	ACTIVE
C005	Khách Hàng Trung	Standard	ACTIVE
C006	Pham Van X	Standard	ACTIVE
C007	Nguyen Thi Y	Standard	ACTIVE
C008	Pham Van D	Gold	ACTIVE
C009	Nguyen Van A (2)	Silver	ACTIVE
C010	Khách Test VIP	VIP	ACTIVE

CÂU LỆNH SQL

```
SELECT c.customer_id,
       c.customer_name,
       c.membership_tier,
       c.status
FROM   Customers c
LEFT JOIN Orders o
      ON c.customer_id = o.customer_id
WHERE  o.customer_id IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers c LEFT JOIN Orders o ON c.customer_id = o.customer_id	LEFT JOIN giữ TẤT CẢ khách hàng, kể cả người chưa có đơn hàng nào.
WHERE o.customer_id IS NULL	Sau LEFT JOIN, khách không có đơn sẽ có cột o.customer_id = NULL. Lọc đúng những khách đó.
SELECT c.customer_id, c.customer_name, c.membership_tier, c.status	Chiếu thêm tier và status để QA phân loại: tài khoản test, tài khoản lỗi, hay khách thật chưa mua?

Giải thích tổng thể

Cùng kỹ thuật anti-join, chiều Customers → Orders.
Trong data mẫu, 7 trong 10 khách chưa có đơn — đây là tài khoản test được thêm để phục vụ các câu lệnh từ Câu 1 đến Câu 10.
Khi làm sạch dữ liệu trước sprint, QA nên chạy câu này và đánh dấu tài khoản nào là test — tránh đưa vào báo cáo làm sai số liệu thật.

Kết quả sau khi query (minh họa)

customer_id	customer_name	membership_tier	status
C004	Khach Hang Ao Bug	Standard	ACTIVE
C005	Khach Hang Trung	Standard	ACTIVE
C006	Pham Van X	Standard	ACTIVE
C007	Nguyen Thi Y	Standard	ACTIVE
C008	Pham Van D	Gold	ACTIVE
C009	Nguyen Van A (2)	Silver	ACTIVE
C010	Khach Test VIP	VIP	ACTIVE

7 khách chưa có đơn — phần lớn là tài khoản test tạo cho Phần 1. Cần đánh dấu để loại khỏi báo cáo sản xuất.

GÓC SOI LỖI CỦA TESTER

Câu này hay bị nhầm với Câu 5 (đơn hàng mò côi). Điểm khác nhau:
(1) **Câu 5:** Orders không có Customers — đơn hàng mò côi, thiếu chủ.
(2) **Câu 17:** Customers không có Orders — khách hàng chưa mua gì.
Hai hướng bổ sung cho nhau: Câu 5 bắt lỗi tầng Orders, Câu 17 bắt lỗi tầng Customers.

18

Kiểm tra trạng thái đơn hàng ngoài danh sách cho phép

TÌNH HUỐNG

Cột **status** trong Orders chỉ nên chứa các giá trị đã định nghĩa: PENDING, COMPLETED, CANCELLED, PROCESSING. Nếu DB không dùng ENUM và tầng ứng dụng thiếu validation, giá trị lạ như 'DONE', 'PAID', 'done' vẫn có thể trôi vào.

Bảng Orders — kiểm tra cột status (không có giá trị lạ):

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24

CÂU LỆNH SQL

```
SELECT order_id,
       customer_id,
       status
FROM   Orders
WHERE  status NOT IN
      ( 'COMPLETED', 'PENDING',
        'CANCELLED', 'PROCESSING' );
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL tải toàn bộ bảng Orders .
WHERE status NOT IN ('COMPLETED', 'PENDING', 'CANCELLED', 'PROCESSING')	NOT IN lọc tất cả giá trị không nằm trong danh sách cho phép. Danh sách phải đồng bộ với enum trong code — thêm/xóa giá trị phải cập nhật cả hai nơi.
SELECT order_id, customer_id, status	Chiếu ra để QA xác nhận giá trị lạ và truy tìm nguồn gốc.

Giải thích tổng thể
Câu lệnh trả về **kết quả rỗng** — đây là trạng thái bình thường, xác nhận tất cả đơn hàng đang có status hợp lệ.
Kết quả rỗng không phải thất bại — đó là confirmation hệ thống đang đúng.
Trong thực tế, câu này nên chạy định kỳ hoặc sau mỗi lần migrate dữ liệu để phát hiện sớm nếu có giá trị lạ xuất hiện.
Câu 9 áp dụng cùng kỹ thuật cho cột membership_tier của Customers.

Kết quả sau khi query (minh họa)

order_id	customer_id	status
----------	-------------	--------

Kết quả rỗng — tất cả đơn hàng có status hợp lệ. Câu này vẫn cần chạy định kỳ: một lần sạch không có nghĩa là mãi mãi sạch.

GÓC SOI LỖI CỦA TESTER

Cảnh báo với **NOT IN + NULL**: nếu có dòng nào status = NULL, dòng đó sẽ **KHÔNG** được trả về bởi NOT IN. Để bắt cả NULL, thêm điều kiện:

WHERE status NOT IN (...) OR status IS NULL

Câu 9 (membership_tier) và Câu 18 (status) là cặp đôi để kiểm tra toàn bộ trường dạng ENUM trong hệ thống.

19

Tổng hợp chi tiêu theo từng khách hàng

TÌNH HUỐNG

QA cần bảng tổng hợp để kiểm tra: khách nào đã mua, bao nhiêu đơn, tổng tiền bao nhiêu. Bảng này là nền để phát hiện bất thường — khách chi tiêu quá cao/thấp, hoặc số đơn không khớp với dữ liệu loyalty.

Bảng Orders — dữ liệu đầu vào để tổng hợp chi tiêu:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24

CÂU LỆNH SQL

```
SELECT c.customer_id,
       c.customer_name,
       COUNT(o.order_id) AS so_don,
       SUM(o.total_amount) AS tong_chi_tieu
FROM   Customers c
JOIN   Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY tong_chi_tieu DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers c JOIN Orders o ON c.customer_id = o.customer_id	INNER JOIN : chỉ lấy khách đã có ít nhất một đơn. Khách chưa mua (C004–C010) sẽ không xuất hiện — dùng LEFT JOIN nếu muốn hiện cả họ với so_don = 0. ORD_004 (C999) cũng bị loại vì C999 không có trong Customers.
GROUP BY c.customer_id, c.customer_name	Gom tất cả đơn hàng của cùng một khách thành một nhóm để tính tổng.
SELECT ..., COUNT, SUM	COUNT(o.order_id) đếm số đơn; SUM(o.total_amount) cộng tổng tiền. Lưu ý: SUM dùng total_amount từ Orders — chưa chắc khớp với tổng items (xem Câu 11).
ORDER BY tong_chi_tieu DESC	Khách chi tiêu nhiều nhất lên đầu — để phát hiện outlier bất thường. Trên production, thêm LIMIT 10–20 để chỉ lấy nhóm khách chi tiêu cao nhất.

Giải thích tổng thể

Câu lệnh không tìm bug trực tiếp mà tạo **bảng nền để phát hiện bất thường**.
QA so sánh kết quả với dữ liệu hệ thống loyalty, CRM hoặc báo cáo tài chính — nếu con số lệch là có vấn đề.
Lưu ý: vì dùng total_amount từ Orders, kết quả bao gồm Bug 1 (ORD_002 ghi sai).
Cần chạy Câu 11 trước để phát hiện và sửa dữ liệu lệch, sau đó Câu 19 mới phản ánh đúng thực tế.

Ket qua sau khi query (minh hoa)

customer_id	customer_name	so_don	tong_chi_tieu
C001	Nguyen Van A	1	32.000.000
C002	Tran Van B	1	20.000.000
C003	Le Thi C	1	8.000.000

Chỉ 3/10 khách có đơn (C001, C002, C003); 7 khách C004–C010 chưa mua nên không xuất hiện. C001 dẫn đầu 32M. Con số C002 (20M) là Bug 1 — thực tế items cộng lại 31M. C003 vẫn được tính dù đơn đã CANCELLED (câu này không lọc status).

GÓC SOI LỖI CỦA TESTER

Câu này dùng total_amount từ Orders — con số ghi sẵn, chưa chắc đúng.
Để tính chính xác từ Order_Items:
Thay **SUM(o.total_amount)** bằng **SUM(oi.quantity * oi.price)** sau khi JOIN thêm bảng Order_Items.
Chạy Câu 11 trước để phát hiện và sửa dữ liệu lệch, rồi mới dùng Câu 19 báo cáo.

20 Phát hiện sản phẩm đã bán vượt quá tồn kho

TÌNH HUỐNG

Tổng số lượng đã bán (từ Order_Items) lớn hơn tồn kho hiện tại (từ Products). Đây là dấu hiệu của race condition, thiếu kiểm tra stock trước khi trừ, hoặc dữ liệu kho bị cập nhật sai sau khi xác nhận đơn.

Bảng Products — dòng đỏ: tồn kho có thể bị vượt quá:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	NULL	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	NULL

CÂU LỆNH SQL

```
SELECT p.product_id,
       p.product_name,
       p.stock,
       SUM(oi.quantity) AS tong_da_ban
FROM   Products p
JOIN   Order_Items oi ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name, p.stock
HAVING SUM(oi.quantity) > p.stock;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products p JOIN Order_Items oi ON p.product_id = oi.product_id	INNER JOIN ghép mỗi sản phẩm với tất cả dòng trong Order_Items. Sản phẩm chưa bán sẽ không xuất hiện — dùng LEFT JOIN + Câu 16 để kiểm tra.
GROUP BY p.product_id, p.product_name, p.stock	Gom tất cả dòng Order_Items của cùng một sản phẩm để tính tổng số đã bán.
HAVING SUM(oi.quantity) > p.stock	HAVING so sánh sau aggregate: tổng đã bán > tồn kho hiện tại là vi phạm ràng buộc nghiệp vụ.
SELECT ..., tong_da_ban	Hiển thị cả stock hiện tại và tổng đã bán để QA thấy ngay mức độ vượt quá.

Giải thích tổng thể

Câu lệnh phát hiện vi phạm ràng buộc: **tổng bán ra không được vượt tồn kho**.
PROD_003 bị phát hiện vì stock = -5 và tong_da_ban = 1 (1 > -5).
Điều này nghĩa là hệ thống đã bán sản phẩm khi tồn kho đã âm — không có check trước khi giảm kho.
Với hệ thống thực, câu này giúp phát hiện overselling trước khi khách hàng phàn nàn vì không nhận được hàng.

Ket qua sau khi query (minh hoa)

product_id	product_name	stock	tong_da_ban
PROD_003	Tai nghe Sony WH-1000XM5	-5	1

PROD_003: tồn kho = -5 nhưng tổng đã bán = 1. Hệ thống đã cho phép bán khi kho đã âm — thiếu validation trước khi trừ kho.

GÓC SOI LỖI CỦA TESTER

INNER JOIN làm câu này bỏ qua sản phẩm chưa có đơn nào.
Ba câu cùng nhau tạo bức tranh đầy đủ về tình trạng kho:

- (1) **Câu 12**: stock âm — phát hiện trực tiếp không cần tính tổng bán.
- (2) **Câu 16**: sản phẩm chưa bán — góc nhìn ngược lại.
- (3) **Câu 20**: tổng bán > tồn kho — phát hiện overselling.

PHẦN 3 — Đối soát và tính toán

Phần lớn bug tài chính không nằm ở một bản ghi đơn lẻ mà ở chỗ hai con số đáng lẽ bằng nhau lại lệch nhau. Nhóm này tập trung vào kỹ thuật đối soát: tổng header so với tổng detail, tồn kho so với lượng đã bán.

21

Tính doanh thu thực theo từng đơn từ Order_Items

TÌNH HUỐNG

Câu 19 tổng hợp chỉ tiêu dùng **total_amount** — con số ghi sẵn trong Orders, có thể sai như Bug 1 ở ORD_002. Doanh thu thật phải tính trực tiếp từ Order_Items: **SUM(quantity × price)**. Câu này là điểm khởi đầu để QA xác minh độ tin cậy của dữ liệu trước khi đưa vào báo cáo.

Bảng Order_Items — dòng đỏ: item_id=7 nhân đôi PROD_001 trong ORD_001:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000

CÂU LỆNH SQL

```
SELECT order_id,
       COUNT(*)           AS so_dong_items,
       SUM(quantity)       AS tong_so_luong,
       SUM(quantity * price) AS doanh_thu_thuc
FROM   Order_Items
GROUP BY order_id
ORDER BY order_id;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items	MySQL tải toàn bộ bảng Order_Items — 6 dòng trong dữ liệu mẫu.
GROUP BY order_id	Gom tất cả dòng có cùng order_id thành một nhóm. Mỗi nhóm đại diện cho một đơn hàng.
COUNT(*), SUM(quantity), SUM(quantity * price)	COUNT(*) đếm số dòng items của đơn. SUM(quantity) tổng số lượng sản phẩm. SUM(quantity × price) tính tiền thực từ items.
ORDER BY order_id	Sắp xếp kết quả theo mã đơn để so sánh dễ hơn với bảng Orders.

Giải thích tổng thể

Kỹ thuật **GROUP BY + aggregate** tổng hợp từ bảng chi tiết lên level đơn hàng.

COUNT(*) là chỉ số cảnh báo sớm: nếu con số lớn bất thường so với dự kiến, rất có thể có item bị trùng.

Trong data mẫu, ORD_001 có 3 dòng thay vì 2 — $COUNT(*) = 3$ là tín hiệu đầu tiên trước khi đối soát doanh_thu_thuc với total_amount.

Kết quả sau khi query (minh họa)

order_id	so_dong_items	tong_so_luong	doanh_thu_thuc
ORD_001	3	3	62.000.000
ORD_002	2	2	31.000.000
ORD_003	1	1	8.000.000

ORD_001: 3 dòng items → COUNT bất thường, doanh thu = 62M (lẽ ra 32M). ORD_002: tổng = 31M, khác total_amount 20M (Bug 1). ORD_004 vắng mặt vì không có item nào.

GÓC SOI LỖI CỦA TESTER

Kết hợp câu này với Câu 11 để phân tích sâu hơn:

(1) **Câu 21**: tính doanh thu thực từ items theo từng đơn — con số tuyệt đối.

(2) **Câu 11**: chỉ lọc ra đơn có doanh_thu_thuc ≠ total_amount — xác định đúng đơn lỗi.

Câu 21 cho bức tranh tổng thể; Câu 11 chỉ đích danh đơn cần điều tra.

22

Xếp hạng sản phẩm bán chạy nhất theo doanh số

TÌNH HUỐNG

Báo cáo sản phẩm bán chạy là cơ sở để ra quyết định nhập hàng và marketing. Nhưng nếu dữ liệu Order_Items có item trùng, bảng xếp hạng sẽ sai — sản phẩm bình thường đột ngột lên top vì được tính nhiều lần.

Bảng Order_Items — dòng đỏ: item_id=7 làm PROD_001 bị tính 3 lần:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000

CÂU LỆNH SQL

```

SELECT p.product_id,
       p.product_name,
       SUM(oi.quantity)           AS tong_so_luong,
       SUM(oi.quantity * oi.price) AS tong_doanh_so
FROM   Products p
JOIN   Order_Items oi
      ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name
ORDER BY tong_doanh_so DESC;

```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products p JOIN Order_Items oi ON p.product_id = oi.product_id	INNER JOIN ghép mỗi sản phẩm với tất cả items bán ra. Sản phẩm chưa bán (PROD_005, 006, 007) bị loại — dùng LEFT JOIN nếu muốn hiện cả chúng với doanh_so = NULL.
GROUP BY p.product_id, p.product_name	Gom tất cả dòng Order_Items của cùng một sản phẩm thành một nhóm để tính tổng số lượng và doanh số.
ORDER BY tong_doanh_so DESC	Sản phẩm doanh số cao nhất nổi lên đầu — dễ phát hiện outlier nếu chênh lệch bất thường.

Giải thích tổng thể

Kỹ thuật **JOIN + GROUP BY + ORDER BY** là nền của mọi báo cáo xếp hạng.

Kết quả hiện tại bị méo vì dữ liệu đầu vào có lỗi: PROD_001 xuất hiện 3 lần trong items (item 1, 4, 7) thay vì 2.

Doanh số thực của PROD_001 chỉ là 60M — con số 90M là ảo do item trùng.

Bảng xếp hạng chính xác phải bắt đầu bằng việc xác nhận dữ liệu sạch (Câu 2).

Ket qua sau khi query (minh hoa)

product_id	product_name	tong_so_luong	tong_doanh_so
PROD_001	iPhone 15 Pro Max	3	90.000.000
PROD_003	Tai nghe Sony WH-1000XM5	1	8.000.000
PROD_002	Ban phim co Logitech	1	2.000.000
PROD_004	Sac du phong Anker	1	1.000.000

PROD_001 dẫn đầu với 90M — thực ra chỉ 60M nếu không có item 7 trùng. Chạy Câu 2 để xác nhận trùng, sửa dữ liệu rồi mới tin vào kết quả xếp hạng này.

GÓC SOI LỖI CỦA TESTER

Trước khi dùng kết quả xếp hạng để ra quyết định kinh doanh, QA cần kiểm tra:

(1) **Câu 2:** có item nào bị trùng (order_id + product_id) không?

(2) **Câu 21:** COUNT(*) theo order có dòng nào bất thường không?

Dữ liệu bản ở Order_Items lan trực tiếp lên báo cáo cấp quản lý — đây là loại bug dễ bị bỏ qua nhất vì không gây lỗi hệ thống.

23 Kiểm tra giá trị tồn kho (stock × price)

TÌNH HUỐNG

Giá trị tồn kho = **stock × price** là chỉ số tài chính quan trọng trong quản lý kho. Nếu stock âm (Bug 2) hoặc price = NULL, giá trị tính ra sẽ bị âm hoặc NULL — báo cáo kế toán sẽ sai lệch.

Bảng Products — dòng đỏ: stock âm hoặc NULL sẽ cho gia_tri_kho sai:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5

product_id	product_name	category	price	stock
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT product_id,
       product_name,
       price,
       stock,
       price * stock AS gia_tri_kho
FROM   Products
ORDER BY gia_tri_kho;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products — 7 sản phẩm trong dữ liệu mẫu.
price * stock AS gia_tri_kho	MySQL tính tích hai cột và đặt tên kết quả là gia_tri_kho . Quy tắc: NULL × bất kỳ = NULL; âm × dương = âm.
ORDER BY gia_tri_kho	Sắp xếp tăng dần: MySQL đặt NULL trước tiên, rồi đến âm, rồi dương. Cách hiển thị này đưa dòng lỗi lên đầu bảng kết quả.

Giải thích tổng thể

Phép nhân đơn giản nhưng phản ánh ngay **hai loại lỗi dữ liệu** từ Câu 12–14:

- (1) **stock âm**: PROD_003 cho gia_tri_kho = -40.000.000 — không thể có kho giá trị âm.
- (2) **NULL**: PROD_006 (price NULL) và PROD_007 (stock NULL) cho gia_tri_kho = NULL — không thể tính được giá trị kho, ảnh hưởng trực tiếp đến báo cáo tổng tài sản.

Ket qua sau khi query (minh hoa)

product_id	product_name	price	stock	gia_tri_kho
PROD_006	Loa Bluetooth JBL	(NULL)	10	(NULL)
PROD_007	Chuot gaming Razer	1.500.000	(NULL)	(NULL)
PROD_003	Tai nghe Sony	8.000.000	-5	-40.000.000
PROD_004	Sac du phong Anker	1.000.000	20	20.000.000
PROD_005	Ban phim co Logitech	2.000.000	30	60.000.000
PROD_002	Ban phim co Logitech	2.000.000	100	200.000.000
PROD_001	iPhone 15 Pro Max	30.000.000	50	1.500.000.000

3 dòng bất thường ở đầu: 2 NULL (không tính được) và 1 âm -40M (tồn kho âm do Bug 2). ORDER BY gia_tri_kho tự động đưa các dòng lỗi lên đầu — không cần WHERE để lọc riêng.

GÓC SOI LỖI CỦA TESTER

Để chỉ lấy dòng lỗi, thêm điều kiện lọc vào câu lệnh:

(1) Kho giá trị âm: **WHERE price * stock < 0**

(2) Kho không tính được: **WHERE price IS NULL OR stock IS NULL**

Câu 23 (kho âm) kết hợp với Câu 12 (stock âm) và Câu 14 (price NULL) tạo thành bộ 3 kiểm tra tồn kho đầy đủ.

24 So sánh giá bán lịch sử với giá niêm yết hiện tại**TÌNH HUỐNG**

Cột **price** trong Order_Items là giá tại thời điểm mua — snapshot của giá khi giao dịch xảy ra. Cột **price** trong Products là giá niêm yết hiện tại. Nếu hai con số khác nhau, có thể bình thường (giá thay đổi) hoặc là bug (ghi nhầm giá lúc tạo đơn). QA cần kiểm tra để phân biệt hai trường hợp.

Bảng Order_Items — so sánh oi.price (lúc mua) với Products.price (hiện tại):

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000

CÂU LỆNH SQL

```
SELECT oi.order_id,
       oi.product_id,
       oi.price     AS gia_luc_ban,
       p.price      AS gia_hien_tai,
       oi.price - p.price AS chenh_lech
FROM   Order_Items oi
JOIN   Products p
      ON oi.product_id = p.product_id
WHERE  oi.price <> p.price;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items oi JOIN Products p ON oi.product_id = p.product_id	INNER JOIN ghép mỗi item với thông tin sản phẩm tương ứng. Kết quả có cả oi.price (lúc mua) và p.price (hiện tại) trên cùng một dòng.
WHERE oi.price <> p.price	Lọc chỉ những dòng có giá bán khác giá hiện tại. Kết quả rỗng = tất cả items được ghi đúng giá.
SELECT ..., oi.price - p.price AS chenh_lech	Tính độ lệch: dương = bán đắt hơn giá hiện tại; âm = bán rẻ hơn giá hiện tại.

Giải thích tổng thể

Order_Items.price là **giá snapshot** — lưu lại giá đúng lúc khách mua, không đổi kể cả khi Products.price thay đổi sau đó.

Kết quả rỗng trong data mẫu là bình thường — giá chưa thay đổi.
Thực tế, câu này quan trọng khi team sửa giá sản phẩm: cần xác nhận đơn cũ đã dùng đúng giá cũ, không phải giá mới bị gán ngược.

Ket qua sau khi query (minh hoa)

order_id	product_id	gia_luc_ban	gia_hien_tai	chenh_lech
----------	------------	-------------	--------------	------------

Kết quả rỗng — tất cả items được ghi đúng giá so với Products hiện tại. Câu này nên chạy lại sau mỗi lần cập nhật bảng giá sản phẩm.

GÓC SOI LỖI CỦA TESTER

Kết quả rỗng ở đây là tốt, nhưng cần phân biệt hai tình huống:

(1) **Chênh lệch hợp lý**: giá sản phẩm tăng/giảm sau khi đơn đã đặt — oi.price đúng, p.price là giá mới. Không phải bug.

(2) **Chênh lệch bất thường**: item vừa tạo đã khác giá niêm yết — có thể bug khi tạo đơn hoặc có discount không được ghi nhận đúng cách.

25

Tổng doanh thu chỉ tính đơn hàng COMPLETED

TÌNH HUỐNG

Doanh thu thật chỉ đến từ đơn đã hoàn tất. Nếu tổng hợp cả đơn CANCELLED hay PENDING, con số sẽ bị thổi phồng. QA cần xác minh hệ thống lọc đúng trạng thái khi tính doanh thu trước khi chốt số báo cáo cuối kỳ.

Bảng Orders — dòng đỏ: ORD_002 có total_amount sai (Bug 1):

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24

CÂU LỆNH SQL

```
SELECT c.customer_name,
       o.order_id,
       o.total_amount,
       o.order_date
FROM   Orders o
JOIN   Customers c
      ON o.customer_id = c.customer_id
WHERE  o.status = 'COMPLETED'
ORDER BY o.total_amount DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

```
FROM Orders o
JOIN Customers c
  ON o.customer_id = c.customer_id
```

INNER JOIN lấy tên khách hàng cho mỗi đơn. ORD_004 (customer_id = C999 không tồn tại) tự động bị loại bởi JOIN này.

WHERE o.status = 'COMPLETED'	Lọc chỉ đơn đã hoàn tất. ORD_003 (CANCELLED) và ORD_004 (PENDING) không được tính vào doanh thu.
ORDER BY o.total_amount DESC	Đơn có giá trị lớn nhất lên đầu — để phát hiện đơn bất thường hoặc Bug 1 khi đối chiếu với Câu 11.

Giải thích tổng thể

Thêm **WHERE status = 'COMPLETED'** là điều kiện bắt buộc trong mọi câu báo cáo doanh thu. Trong data mẫu, tổng hai đơn COMPLETED = 32M + 20M = 52M. Nhưng ORD_002 ghi total_amount = 20M trong khi SUM(items) = 31M (Bug 1) — doanh thu thực đáng ra là 63M.

Ket qua sau khi query (minh hoa)

customer_name	order_id	total_amount	order_date
Nguyen Van A	ORD_001	32.000.000	2026-06-20
Tran Van B	ORD_002	20.000.000	2026-06-22

2 đơn COMPLETED, tổng 52M. Con số của ORD_002 (20M) bị sai do Bug 1 — chạy Câu 11 để xác nhận trước khi chốt báo cáo.

GÓC SOI LỖI CỦA TESTER

Câu này chỉ lọc theo status — không kiểm tra total_amount có đúng không. Để có doanh thu chính xác nhất, dùng tổ hợp ba câu:

(1) **Câu 11**: phát hiện đơn có total_amount lệch với tổng items.

(2) **Câu 25**: lọc chỉ đơn COMPLETED.

(3) **Câu 21**: tính lại doanh thu từ items cho những đơn đã xác nhận sạch.

26

Phân tích tỷ lệ đơn hàng theo trạng thái

TÌNH HUỐNG

Tỷ lệ CANCELLED cao là dấu hiệu vấn đề UX, luồng thanh toán, hoặc bug nghiệp vụ. QA cần bảng thống kê trạng thái đơn để theo dõi xu hướng theo sprint và phát hiện khi tỷ lệ một trạng thái thay đổi bất thường.

Bảng Orders — dữ liệu đầu vào để phân tích trạng thái:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24

CÂU LỆNH SQL

```
SELECT status,
       COUNT(*) AS so_don,
       ROUND(COUNT(*) * 100.0 /
             (SELECT COUNT(*) FROM Orders), 1)
       AS phan_tram
FROM   Orders
GROUP BY status
ORDER BY so_don DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL tải toàn bộ bảng Orders .
GROUP BY status	Gom tất cả đơn có cùng trạng thái thành một nhóm. Kết quả là 1 dòng cho mỗi giá trị status tồn tại trong bảng.
COUNT(*) * 100.0 / (SELECT COUNT(*) FROM Orders)	Subquery đếm tổng số đơn. Chia COUNT nhóm cho tổng rồi nhân 100 để ra phần trăm. ROUND(..., 1) làm tròn 1 chữ số thập phân.
ORDER BY so_don DESC	Trạng thái phổ biến nhất lên đầu.

Giải thích tổng thể

Subquery (**SELECT COUNT(*) FROM Orders**) trong SELECT là scalar subquery — trả về một giá trị dùng làm mẫu số cho tất cả các dòng.

Nhân với **100.0** (không phải 100) để ép MySQL dùng phép chia số thực — nếu dùng 100 (số nguyên), kết quả sẽ bị làm tròn xuống 0.

Với data mẫu: COMPLETED = 50%, mỗi trạng thái còn lại = 25%.

Kết quả sau khi query (minh họa)

status	so_don	phan_tram
COMPLETED	2	50.0
CANCELLED	1	25.0
PENDING	1	25.0

50% đơn hoàn tất, 25% bị hủy. Với dữ liệu test ít, tỷ lệ này không phản ánh thực tế. Câu này có giá trị thật khi chạy trên dữ liệu production đủ lớn.

GÓC SOI LỖI CỦA TESTER

Hai lưu ý khi dùng câu này trên production:

- (1) **Trạng thái lạ**: nếu xuất hiện status không trong danh sách chuẩn, đây là dòng bất thường cần điều tra — kết hợp với Câu 18.
- (2) **Tỷ lệ thay đổi đột ngột**: CANCELLED tăng từ 10% lên 30% sau một sprint là tín hiệu cần xem lại luồng checkout hoặc thanh toán.

27 Tổng doanh thu theo danh mục sản phẩm

TÌNH HUỐNG

Quản lý sản phẩm cần biết danh mục nào đóng góp bao nhiêu vào doanh thu để ra quyết định nhập hàng và phân bổ ngân sách. Câu lệnh này tổng hợp từ Order_Items lên level danh mục qua bảng Products.

Bảng Order_Items — dòng đỏ: item_id=7 làm danh mục Điện thoại bị thổi phồng:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000

CÂU LỆNH SQL

```
SELECT p.category,
       COUNT(DISTINCT oi.order_id) AS so_don_co_sp,
       SUM(oi.quantity)           AS tong_so_luong,
       SUM(oi.quantity * oi.price) AS tong_doanh_so
FROM   Order_Items oi
JOIN   Products p
      ON oi.product_id = p.product_id
GROUP BY p.category
ORDER BY tong_doanh_so DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items oi JOIN Products p ON oi.product_id = p.product_id	INNER JOIN lấy thông tin category từ Products cho mỗi dòng Order_Items. Sản phẩm chưa bán không xuất hiện — danh mục chỉ tồn tại nếu có đơn.
GROUP BY p.category	Gom tất cả items về cùng danh mục thành một nhóm. Mỗi dòng kết quả = một danh mục.
COUNT(DISTINCT oi.order_id), SUM(oi.quantity * oi.price)	COUNT(DISTINCT order_id) đếm số đơn có ít nhất 1 sản phẩm thuộc danh mục này (không tính trùng). SUM(quantity × price) tính tổng doanh số.
ORDER BY tong_doanh_so DESC	Danh mục doanh số cao nhất lên đầu.

Giải thích tổng thể

Kỹ thuật **JOIN + GROUP BY category** tổng hợp dữ liệu qua hai bảng lên một level cao hơn. Kết quả bị ảnh hưởng bởi item trùng: 'Điện thoại' cho 90M thay vì 60M thực tế. **COUNT(DISTINCT order_id)** thay vì **COUNT(*)**: nếu một đơn có 2 item cùng danh mục, đơn đó chỉ được đếm 1 lần — phản ánh đúng số đơn có mua sản phẩm danh mục này.

Kết quả sau khi query (minh họa)

category	so_don_co_sp	tong_so_luong	tong_doanh_so
Điện thoại	2	3	90.000.000
Phụ kiện	3	3	11.000.000

'Điện thoại' dẫn đầu 90M — bị thổi phồng do item 7 trùng. Thực tế là 60M. 'Phụ kiện' 11M từ 3 đơn khác nhau (PROD_002, 004, 003).

GÓC SOI LỖI CỦA TESTER

Kết quả chỉ có 2 danh mục vì INNER JOIN loại sản phẩm chưa bán.

Nếu muốn xem đầy đủ tất cả danh mục (kể cả chưa có doanh số):

Đổi sang **RIGHT JOIN** Products rồi LEFT JOIN Order_Items — danh mục chưa bán sẽ hiện với tong_doanh_so = NULL.

Cũng cần chạy Câu 2 để làm sạch item trùng trước khi dùng câu này báo cáo.

28

Đối soát tồn kho hiện tại với lượng đã bán ra

TÌNH HUỐNG

Tồn kho hiện tại (stock) + lượng đã bán (SUM items) phải khớp với tồn kho ban đầu. Nếu con số ước tính không hợp lý — âm, quá thấp hoặc quá cao — có thể có giao dịch bị bỏ sót, dữ liệu kho bị sửa thủ công, hoặc bug trong logic trừ kho.

Bảng Products — dòng đỏ: PROD_003 stock âm sẽ cho uoc_tinh_ban_dau bất thường:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuat gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT p.product_id,
       p.product_name,
       p.stock AS ton_kho_hien_tai,
       COALESCE(SUM(oi.quantity), 0) AS tong_da_ban,
       p.stock
       + COALESCE(SUM(oi.quantity), 0)
       AS uoc_tinh_ban_dau
FROM   Products p
LEFT JOIN Order_Items oi
      ON p.product_id = oi.product_id
GROUP BY p.product_id, p.product_name, p.stock
ORDER BY p.product_id;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products p LEFT JOIN Order_Items oi ON p.product_id = oi.product_id	LEFT JOIN giữ TẤT CẢ sản phẩm, kể cả chưa bán. Sản phẩm chưa bán sẽ có SUM(oi.quantity) = NULL từ phía Order_Items.
COALESCE(SUM(oi.quantity), 0) AS tong_da_ban	COALESCE chuyển NULL thành 0 cho sản phẩm chưa bán. Không dùng COALESCE thì tong_da_ban = NULL — phép cộng tiếp theo sẽ bị NULL.

p.stock + COALESCE(SUM(oi.quantity), 0) AS uoc_tinh_ban_dau	Ước tính tồn kho ban đầu = tồn hiện tại + đã bán. Nếu con số này âm → có giao dịch khi kho đã âm.
GROUP BY p.product_id, p.product_name, p.stock	p.stock phải có trong GROUP BY vì được dùng trong SELECT nhưng không phải aggregate.

Giải thích tổng thể

Kỹ thuật **LEFT JOIN + COALESCE + aggregate** tạo bảng đối soát kho đầy đủ.
PROD_003: uoc_tinh_ban_dau = -5 + 1 = **-4** — tồn kho ước tính ban đầu âm, vật lý không thể xảy ra, xác nhận kho bị ghi sai trước khi có đơn hàng.
PROD_001: 50 + 3 = 53 — lẽ ra phải là 52 nếu chỉ bán 2 unit, chênh 1 đơn vị chính xác bằng item 7 trùng.

Ket qua sau khi query (minh hoa)

product_id	product_name	ton_kho_hien_tai	tong_da_ban	uoc_tinh_ban_dau
PROD_001	iPhone 15 Pro Max	50	3	53
PROD_002	Ban phim co Logitech	100	1	101
PROD_003	Tai nghe Sony	-5	1	-4
PROD_004	Sac du phong Anker	20	1	21
PROD_005	Ban phim co Logitech	30	0	30
PROD_006	Loa Bluetooth JBL	10	0	10
PROD_007	Chuot gaming Razer	(NULL)	0	(NULL)

PROD_003: uoc_tinh_ban_dau = -4 → bất khả thi, xác nhận kho bị ghi âm từ trước khi bán. PROD_001: 53 thay vì 52 → item trùng thổi phồng thêm 1 đơn vị.

GÓC SOI LỖI CỦA TESTER

COALESCE là hàm quan trọng khi LEFT JOIN: không dùng sẽ cho NULL không tính được.

Hai trường hợp kết quả bất thường cần điều tra thêm:

(1) **uoc_tinh_ban_dau âm**: kho đã âm trước khi có đơn — bug dữ liệu gốc.

(2) **uoc_tinh_ban_dau quá cao**: có thể items bị trùng hoặc kho không trừ đúng.

PROD_007 trả về NULL vì stock = NULL — COALESCE chỉ xử lý phía tong_da_ban, không xử lý p.stock.

29 **Phát hiện đơn hàng bị tạo trùng (double order)**

TÌNH HUỐNG

Khi người dùng bấm nút Đặt hàng nhiều lần hoặc xảy ra lỗi mạng khiến request gửi hai lần, hệ thống có thể tạo hai đơn giống nhau trong tích tắc. Đây là lỗi double-charge nghiêm trọng — khách bị trừ tiền hai lần cho cùng một đơn mà không hay biết.

Bảng Orders — kiểm tra có cặp đơn nào trùng customer + tổng tiền + ngày không:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24

CÂU LỆNH SQL

```
SELECT customer_id,
       total_amount,
       order_date,
       COUNT(*) AS so_lan
FROM   Orders
GROUP BY customer_id, total_amount, order_date
HAVING COUNT(*) > 1;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL tải toàn bộ bảng Orders .
GROUP BY customer_id, total_amount, order_date	Gom theo bộ ba: cùng khách, cùng tổng tiền, cùng ngày. Nếu một nhóm có nhiều hơn 1 đơn — rất có thể là double order.
HAVING COUNT(*) > 1	Chỉ giữ nhóm xuất hiện từ 2 lần trở lên — dấu hiệu của đơn bị tạo trùng.

Giải thích tổng thể

Kỹ thuật **GROUP BY nhiều cột + HAVING COUNT** — cùng mẫu với Câu 1 và Câu 2, áp dụng cho bảng Orders để phát hiện double submission.
Kết quả rỗng trong data mẫu là bình thường — chưa có đơn trùng.
Trong thực tế, câu này nên chạy sau mỗi đợt stress test hoặc khi khách phản ánh bị trừ tiền hai lần.

Ket qua sau khi query (minh họa)

customer_id	total_amount	order_date	so_lan
-------------	--------------	------------	--------

Kết quả rỗng — không có đơn hàng bị tạo trùng. Chạy lại câu này sau stress test để phát hiện sớm double-charge bug.

GÓC SOI LỖI CỦA TESTER

Gộp theo **order_date** (ngày) có thể bỏ sót double order xảy ra trong cùng ngày nhưng cách nhau vài phút — vẫn là hai đơn hợp lệ.
Nếu DB lưu cả giờ phút, dùng điều kiện chặt hơn:
TIMESTAMPDIFF(MINUTE, o1.order_date, o2.order_date) < 5 — chỉ bắt hai đơn cách nhau dưới 5 phút mới là double order thật sự.

30 Phát hiện đơn hàng có giá trị bất thường (outlier)

TÌNH HUỐNG

Đơn hàng có total_amount quá cao so với mức bình thường có thể do: nhập nhầm số lượng, giá sản phẩm bị set sai, hoặc bug logic tính tiền. Câu lệnh này tìm đơn vượt ngưỡng 1.5× trung bình như một bước kiểm tra nhanh.

Bảng Orders — dòng đỏ: ORD_001 vượt ngưỡng 1.5× trung bình:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24

CÂU LỆNH SQL

```
SELECT order_id,
       customer_id,
       total_amount
FROM   Orders
WHERE  total_amount >
      (SELECT AVG(total_amount) * 1.5
       FROM Orders)
ORDER BY total_amount DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL tải toàn bộ bảng Orders .
WHERE total_amount > (SELECT AVG(total_amount) * 1.5 FROM Orders)	Subquery tính trung bình total_amount rồi nhân 1.5 làm ngưỡng. Trong data mẫu: AVG = 16.250.000 → ngưỡng = 24.375.000. Chỉ ORD_001 (32M) vượt ngưỡng này.
ORDER BY total_amount DESC	Đơn có giá trị cao nhất lên đầu để QA xem xét trước.

Giải thích tổng thể

Dùng **subquery trong WHERE** để so sánh từng dòng với một ngưỡng động tính từ chính bảng đó — không cần biết trước ngưỡng là bao nhiêu.
AVG trong data mẫu = 16.250.000 → ngưỡng 1.5× = 24.375.000.
ORD_001 (32M) vượt ngưỡng và được flag. Điều này không có nghĩa là lỗi — đây là điểm khởi đầu để QA điều tra thêm bằng Câu 11.

Ket qua sau khi query (minh họa)

order_id	customer_id	total_amount
ORD_001	C001	32.000.000

ORD_001 (32M) vượt ngưỡng 1.5× trung bình (24.375.000). Tổng tiền 32M là hợp lệ (30M + 2M), nhưng doanh thu từ items = 62M do item trùng — đây mới là bất thường thật sự, cần Câu 11 để phát hiện.

GÓC SOI LỖI CỦA TESTER

Ngưỡng 1.5× trung bình là điểm khởi đầu — điều chỉnh tùy nghiệp vụ:

- (1) Đơn B2B thường có giá trị lớn hơn B2C nhiều lần — nên tách hai nhóm riêng.
- (2) Nếu muốn chặt hơn, dùng **2×** hoặc **3×** trung bình.
- (3) Phương pháp thống kê tốt hơn là dùng **standard deviation**: flag đơn vượt trung_bình + 2×stddev — nhưng MySQL không có hàm STDDEV trực tiếp trong HAVING, cần dùng subquery.

PHẦN 4 — Biên và dữ liệu bất thường

Người dùng thật luôn nhập những thứ ngoài dự đoán: chuỗi quá dài, ký tự lạ, số âm, ngày tháng vô lý. Đây là nhóm câu lệnh để tìm những outlier mà form nhập liệu lẽ ra phải chặn từ đầu.

31

Tìm tên khách hàng chứa ký tự bất thường

TÌNH HUỐNG

Form nhập liệu thường chỉ chặn ký tự đặc biệt ở frontend. Nếu backend API không validate, tên khách có thể chứa ký tự số, ngoặc, ký tự lạ — gây lỗi khi render, export PDF, hoặc tích hợp hệ thống bên ngoài vốn kỳ vọng dữ liệu sạch.

Bảng Customers — dòng đỏ: tên chứa ký tự ngoài chữ cái thường:

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name
FROM   Customers
WHERE  customer_name REGEXP '[0-9()]';
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
WHERE customer_name REGEXP '[0-9()]'	REGEXP kiểm tra chuỗi theo mẫu biểu thức chính quy. Pattern [0-9()] khớp với bất kỳ ký tự nào là chữ số (0–9) hoặc dấu ngoặc tròn.
SELECT customer_id, customer_name	Chiếu hai cột để QA xác minh và quyết định chuẩn hóa.

Giải thích tổng thể

REGEXP (Regular Expression) là công cụ mạnh để kiểm tra định dạng dữ liệu mà LIKE không làm được. Pattern **[0-9()]** khớp với CHỮ SỐ hoặc DẤU NGOẶC — hai loại ký tự không nên xuất hiện trong tên người thật. C009 'Nguyen Van A (2)' bị bắt vì có cả '(' ')' và chữ số '2' — đây là tên được nhập để đối phó với ràng buộc UNIQUE, không phải tên thật.

Ket qua sau khi query (minh hoa)

customer_id	customer_name
C009	Nguyen Van A (2)

1 bản ghi: C009 có tên chứa ngoặc và số — dấu hiệu dữ liệu test hoặc người dùng cố gắng bypass ràng buộc unique tên.

GÓC SOI LỖI CỦA TESTER

Mở rộng pattern để bắt thêm ký tự đặc biệt khác:

(1) **[!@#\$\$%^&*]**: bắt ký tự đặc biệt thường gặp trong SQL injection.

(2) **[^a-zA-Z]**: chỉ cho phép chữ cái Latin và dấu cách — cẩn thận vì sẽ bắt cả tên tiếng Việt có dấu.

Với dữ liệu tiếng Việt, cách an toàn hơn là **whitelist**: kiểm tra từng dòng thủ công thay vì dùng REGEXP.

32

Tìm email không đúng định dạng cơ bản

TÌNH HUỐNG

Câu 3 đã bắt email NULL và chuỗi rỗng. Câu này đi thêm một bước: kiểm tra email **có nội dung** nhưng thiếu ký tự @ hoặc dấu chấm domain — dấu hiệu nhập sai hoặc bypass validation bằng cách nhập chữ bừa.

Bảng Customers — dòng đỏ: email NULL, rỗng hoặc không có @/dấu chấm:

customer_id	customer_name	email	status
C001	Nguyen Van A	a.nguyen@email.com	ACTIVE
C002	Tran Van B	b.tran@email.com	ACTIVE
C003	Le Thi C	c.le@email.com	ACTIVE
C004	Khach Hang Ao Bug	trung_email@email.com	ACTIVE
C005	Khach Hang Trung	trung_email@email.com	ACTIVE
C006	Pham Van X	(NULL)	ACTIVE
C007	Nguyen Thi Y		ACTIVE
C008	Pham Van D	d.pham@email.com	ACTIVE
C009	Nguyen Van A (2)	A.NGUYEN@EMAIL.COM	ACTIVE
C010	Khach Test VIP	vip@email.com	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name,
       email
FROM   Customers
WHERE  email IS NULL
       OR TRIM(email) = ''
       OR email NOT LIKE '%@%'
       OR email NOT LIKE '%.%';
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
WHERE email IS NULL OR TRIM(email) = ''	Hai điều kiện này trùng với Câu 3 — nhắc lại ở đây để câu lệnh hoạt động độc lập, không cần chạy thêm câu khác.
OR email NOT LIKE '%@%' OR email NOT LIKE '%.%'	NOT LIKE '%@%' : email không chứa ký tự @ — không thể hợp lệ. NOT LIKE '%.%' : không có dấu chấm — thiếu phần domain (vd .com).
SELECT customer_id, customer_name, email	Chiếu đủ thông tin để QA liên hệ xác nhận email thật.

Giải thích tổng thể
Câu này kiểm tra email theo **ba tầng**:
(1) Tầng tồn tại: NULL hoặc chuỗi rỗng → không có email.
(2) Tầng cấu trúc: thiếu @ → không thể là email hợp lệ.
(3) Tầng domain: thiếu dấu chấm → không có phần .com/.vn.
LIKE '%@%.%' không bắt được email như 'abc@' hay '@domain.com' — với dữ liệu nghiêm ngặt, dùng REGEXP thay thế.

Ket qua sau khi query (minh hoa)

customer_id	customer_name	email
C006	Pham Van X	(NULL)
C007	Nguyen Thi Y	

C006: email NULL — chưa có. C007: email rỗng — nhập bỏ qua. Các email còn lại đều qua được bộ lọc cơ bản này.

GÓC SOI LỖI CỦA TESTER

LIKE là kiểm tra nhanh nhưng lỏng lẻo — không bắt được các trường hợp như:

(1) 'abc@': có @ nhưng không có domain → LIKE '%@%' vẫn pass.

(2) '@domain.com': thiếu phần local → LIKE cũng pass.

Để kiểm tra chặt hơn, dùng REGEXP:

WHERE email NOT REGEXP '^[A-Za-z0-9._%+\-]+@[A-Za-z0-9.\-]+\.[A-Za-z]{2,}\$'

33

Kiểm tra số lượng sản phẩm bất thường trong Order_Items

TÌNH HUỐNG

Cột quantity phải luôn >= 1. Quantity = 0 là đơn trống về số lượng nhưng vẫn tồn tại trong DB — lỗi logic. Quantity âm có thể là bug trừ kho ngược chiều hoặc dữ liệu hoàn trả bị ghi sai bảng. Quantity quá lớn (> 1000) cũng cần xem xét.

Bảng Order_Items — kiểm tra quantity có trong ngưỡng hợp lệ không:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000

item_id	order_id	product_id	quantity	price
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000

CÂU LỆNH SQL

```
SELECT item_id,
       order_id,
       product_id,
       quantity
FROM   Order_Items
WHERE  quantity <= 0
       OR quantity > 1000;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items	MySQL tải toàn bộ bảng Order_Items .
WHERE quantity <= 0 OR quantity > 1000	Hai điều kiện biên: <= 0 bắt lỗi âm và zero; > 1000 bắt số lượng bất thường lớn (ngưỡng tùy ngành hàng).
SELECT item_id, order_id, product_id, quantity	Chiều đủ thông tin để QA xác định item nào vi phạm và thuộc đơn nào.

Giải thích tổng thể

Kiểm tra biên dưới (**quantity <= 0**) và biên trên (**quantity > 1000**) là cặp kiểm tra chuẩn cho mọi trường số lượng.

Kết quả rỗng trong data mẫu — tất cả items đều có quantity = 1, hoàn toàn hợp lệ.

Kết quả rỗng ở đây là confirmation tốt: hệ thống chưa có bug quantity trong dữ liệu hiện tại.

Kết quả sau khi query (minh họa)

item_id	order_id	product_id	quantity
---------	----------	------------	----------

Kết quả rỗng — tất cả quantity đều hợp lệ (= 1). Chạy lại sau mỗi lần import hàng loạt hoặc sau sprint có tính năng 'hoàn hàng'.

GÓC SOI LỖI CỦA TESTER

Ngưỡng > 1000 chỉ là ví dụ — điều chỉnh tùy nghiệp vụ:

(1) Bán lẻ (B2C): quantity > 10 đã đáng ngờ.

(2) Bán buôn (B2B): quantity > 10.000 mới bất thường.

Để thêm CHECK constraint vào DB ngăn từ đầu:

ALTER TABLE Order_Items ADD CONSTRAINT chk_qty CHECK (quantity >= 1);

34

Kiểm tra ngày đặt hàng bất thường (tương lai hoặc quá xa quá khứ)

TÌNH HUỐNG

Ngày đặt hàng trong tương lai là dấu hiệu đồng hồ server sai, dữ liệu được nhập thủ công với ngày sai, hoặc bug timezone. Ngày quá xa quá khứ (trước 2020) gợi ý dữ liệu migration bị lỗi hoặc giá trị placeholder chưa được cập nhật.

Bảng Orders — kiểm tra order_date có trong khoảng hợp lý không:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24

CÂU LỆNH SQL

```
SELECT order_id,
       customer_id,
       order_date
FROM   Orders
WHERE  order_date > CURDATE()
       OR order_date < '2020-01-01';
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL tải toàn bộ bảng Orders .
WHERE order_date > CURDATE() OR order_date < '2020-01-01'	CURDATE() trả về ngày hiện tại của server — không cần hardcode ngày vào câu lệnh. Ngày < 2020-01-01 là ngưỡng tự chọn; điều chỉnh tùy thời điểm hệ thống bắt đầu hoạt động.
SELECT order_id, customer_id, order_date	Chiếu ngày để QA xác minh nguồn gốc dữ liệu.

Giải thích tổng thể

Dùng **CURDATE()** thay vì hardcode ngày giúp câu lệnh luôn đúng mà không cần sửa theo thời gian.

Kết quả rỗng trong data mẫu — tất cả đơn đều có ngày hợp lệ trong tháng 6/2026.

Trên production, câu này nên chạy định kỳ — đặc biệt sau khi deploy tính năng mới có liên quan đến xử lý thời gian hoặc timezone.

Kết quả sau khi query (minh họa)

order_id	customer_id	order_date
----------	-------------	------------

Kết quả rỗng — tất cả ngày đặt hàng đều hợp lệ. Điều chỉnh ngưỡng '2020-01-01' theo ngày go-live thực tế của hệ thống.

GÓC SOI LỖI CỦA TESTER

Cạm bẫy timezone: **CURDATE()** trả về ngày theo timezone của MySQL server.

Nếu app chạy ở timezone khác (vd UTC+7), ngày 2026-06-24 23:30 UTC+7 = 2026-06-24 16:30 UTC — cùng ngày nhưng **CURDATE()** của server UTC trả về 2026-06-24.

Khi nghi ngờ timezone gây lỗi, thêm điều kiện: **TIMESTAMPDIFF(HOUR, order_date, NOW()) < -8** để bắt đơn 'tương lai' trong vòng 8 giờ.

35 Tìm tên sản phẩm trùng sau khi chuẩn hóa

TÌNH HUỐNG

Câu 8 đã bắt trùng chính xác theo tên và giá. Câu này đi sâu hơn: chuẩn hóa tên về chữ thường và cắt khoảng trắng trước khi so sánh. Bắt được cả các kiểu trùng tinh vi hơn mà **UNIQUE** constraint case-insensitive chưa chặn được.

Bảng Products — dòng đỏ: tên trùng sau khi **LOWER + TRIM**:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT LOWER(TRIM(product_name)) AS ten_chuan,  
       COUNT(*)                AS so_ban_ghi  
FROM   Products  
GROUP BY LOWER(TRIM(product_name))  
HAVING COUNT(*) > 1;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products	MySQL tải toàn bộ bảng Products .
GROUP BY LOWER(TRIM(product_name))	LOWER chuyển về chữ thường, TRIM cắt khoảng trắng hai đầu. Nhờ vậy 'Ban phim co Logitech' và 'ban phim co logitech' rơi vào cùng một nhóm.
HAVING COUNT(*) > 1	Chỉ giữ nhóm có nhiều hơn 1 bản ghi — tên bị trùng sau chuẩn hóa.

Giải thích tổng thể

Kỹ thuật **LOWER + TRIM + GROUP BY** là cách chuẩn hóa trước khi so sánh — đã dùng ở Câu 6 (email), áp dụng tương tự cho product_name.
PROD_002 và PROD_005 đều là 'Ban phim co Logitech' → sau LOWER+TRIM cho 'ban phim co logitech' → COUNT = 2.
Câu 8 đã bắt cặp này theo (tên + giá), câu này bắt lại chỉ theo tên — phát hiện trùng ngay cả khi giá đã bị sửa khác nhau.

Ket qua sau khi query (minh hoa)

ten_chuan	so_ban_ghi
ban phim co logitech	2

'ban phim co logitech' xuất hiện 2 lần (PROD_002 + PROD_005). Chạy thêm SELECT * FROM Products WHERE LOWER(TRIM(product_name)) = 'ban phim co logitech' để xem chi tiết cả hai dòng.

GÓC SOI LỖI CỦA TESTER

Kết quả câu này chỉ cho tên chuẩn hóa và số lần — không biết product_id nào.
Để xem đầy đủ thông tin hai sản phẩm trùng, kết hợp với subquery:

```
SELECT * FROM Products  
WHERE LOWER(TRIM(product_name)) IN  
(SELECT LOWER(TRIM(product_name)) FROM Products  
GROUP BY LOWER(TRIM(product_name)) HAVING COUNT(*) > 1)
```

36

Kiểm tra status của Customers ngoài danh sách cho phép

TÌNH HUỐNG

Câu 9 và Câu 18 đã kiểm tra membership_tier và order status. Cột **status** của Customers cũng cần kiểm tra tương tự: chỉ nên chứa ACTIVE, INACTIVE, SUSPENDED. Giá trị lạ như 'active' (viết thường) hoặc 'BLOCKED' có thể phá vỡ logic phân quyền ứng dụng.

Bảng Customers — kiểm tra cột status (tất cả đang ACTIVE):

customer_id	customer_name	membership_tier	status
C001	Nguyen Van A	Silver	ACTIVE
C002	Tran Van B	Standard	ACTIVE
C003	Le Thi C	Gold	ACTIVE
C004	Khach Hang Ao Bug	Standard	ACTIVE
C005	Khach Hang Trung	Standard	ACTIVE
C006	Pham Van X	Standard	ACTIVE
C007	Nguyen Thi Y	Standard	ACTIVE
C008	Pham Van D	Gold	ACTIVE
C009	Nguyen Van A (2)	Silver	ACTIVE
C010	Khach Test VIP	VIP	ACTIVE

CÂU LỆNH SQL

```
SELECT customer_id,
       customer_name,
       status
FROM   Customers
WHERE  status NOT IN
      ('ACTIVE', 'INACTIVE', 'SUSPENDED');
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers	MySQL tải toàn bộ bảng Customers .
WHERE status NOT IN ('ACTIVE', 'INACTIVE', 'SUSPENDED')	NOT IN lọc ra mọi giá trị không thuộc danh sách cho phép. Danh sách phải được đồng bộ với enum trong application code.
SELECT customer_id, customer_name, status	Chiếu đủ thông tin để QA xác định tài khoản bị ảnh hưởng.

Giải thích tổng thể

- Ba câu 9, 18, và 36 tạo thành bộ kiểm tra ENUM hoàn chỉnh cho hệ thống:
- (1) **Câu 9:** membership_tier của Customers — phát hiện 'VIP' không hợp lệ.
 - (2) **Câu 18:** status của Orders — xác nhận chỉ có 4 trạng thái chuẩn.
 - (3) **Câu 36:** status của Customers — kết quả rỗng, dữ liệu sạch.
- Kết quả rỗng ở đây là confirmation tốt — nhưng vẫn cần chạy định kỳ.

Ket qua sau khi query (minh họa)

customer_id	customer_name	status
-------------	---------------	--------

Kết quả rỗng — tất cả 10 khách hàng đều có status = ACTIVE. Cộng với Câu 9 phát hiện C010 có membership_tier = VIP không hợp lệ: status sạch, tier bẩn.

GÓC SOI LỖI CỦA TESTER

Lưu ý: tất cả khách đều là ACTIVE trong data mẫu — chưa kiểm thử được luồng deactivate/suspend tài khoản.

Khi viết test case, cần thêm test data có đủ các trạng thái:

(1) INACTIVE: tài khoản tự hủy hoặc không hoạt động lâu.

(2) SUSPENDED: bị khóa bởi admin vì vi phạm.

Thiếu data đa dạng = test coverage không đầy đủ.

37 Kiểm tra giá bán âm hoặc bằng 0 trong Order_Items

TÌNH HUỐNG

Câu 14 đã kiểm tra price trong bảng Products. Câu này kiểm tra **price trong Order_Items** — giá tại thời điểm mua. Nếu giá bán = 0 hoặc âm bị ghi vào đơn, khách hàng được mua miễn phí hoặc hệ thống hoàn tiền nhiều hơn số đã thu.

Bảng Order_Items — kiểm tra price có hợp lệ không (tất cả > 0):

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000

CÂU LỆNH SQL

```
SELECT item_id,
       order_id,
       product_id,
       price
FROM   Order_Items
WHERE  price <= 0;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items	MySQL tải toàn bộ bảng Order_Items .
WHERE price <= 0	Lọc item có giá âm hoặc bằng 0. Không cần kiểm tra IS NULL vì cột price trong Order_Items được định nghĩa NOT NULL trong schema.
SELECT item_id, order_id, product_id, price	Chiều đủ thông tin để QA truy ngược: item nào, thuộc đơn nào, sản phẩm gì.

Giải thích tổng thể

Câu 14 kiểm tra Products.price (giá niêm yết), câu này kiểm tra Order_Items.price (giá đã bán).

Hai cột này có thể khác nhau (xem Câu 24) — nên cần kiểm tra độc lập.

Kết quả rỗng trong data mẫu — tất cả giá bán đều hợp lệ.

Trên production, câu này đặc biệt quan trọng sau khi deploy tính năng coupon, flash sale, hoặc discount.

Kết quả sau khi query (minh họa)

item_id	order_id	product_id	price
---------	----------	------------	-------

Kết quả rỗng — tất cả price trong Order_Items đều > 0. Chạy lại sau mỗi sprint có tính năng liên quan đến giá hoặc khuyến mãi.

GÓC SOI LỖI CỦA TESTER

Khác với Products.price có thể NULL, Order_Items.price là NOT NULL trong schema.

Nhưng NOT NULL không có nghĩa là > 0 — chỉ nghĩa là không được để trống.

Để bảo vệ toàn diện, thêm CHECK constraint:

ALTER TABLE Order_Items ADD CONSTRAINT chk_item_price CHECK (price > 0);

38

Phát hiện đơn có tổng tiền nhưng không có sản phẩm nào

TÌNH HUỐNG

Câu 15 đã tìm đơn rỗng (không có items). Câu này thêm điều kiện: đơn có **total_amount > 0** nhưng lại không có item nào — nghĩa là hệ thống đã thu tiền (hoặc ghi nhận số tiền) cho một đơn không có sản phẩm cụ thể.

Đây là mâu thuẫn dữ liệu nghiêm trọng.

Bảng Orders — dòng đỏ: ORD_004 ghi 5M nhưng Order_Items không có dòng nào:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.customer_id,
       o.total_amount,
       o.status
FROM   Orders o
LEFT JOIN Order_Items oi
      ON o.order_id = oi.order_id
WHERE  o.total_amount > 0
      AND oi.order_id IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o LEFT JOIN Order_Items oi ON o.order_id = oi.order_id	LEFT JOIN giữ tất cả đơn hàng. Đơn không có item sẽ có oi.order_id = NULL sau LEFT JOIN.
WHERE o.total_amount > 0 AND oi.order_id IS NULL	Kết hợp hai điều kiện: total_amount > 0 (có ghi nhận tiền) VÀ oi.order_id IS NULL (không có items). Mâu thuẫn này là bug nghiêm trọng.

```
SELECT o.order_id, o.customer_id,
       o.total_amount, o.status
```

Chiều đủ thông tin để QA điều tra: bao nhiêu tiền, trạng thái gì.

Giải thích tổng thể

Câu 15 bắt **mọi đơn rỗng** (kể cả `total_amount = 0`). Câu 38 chỉ bắt đơn rỗng **nhưng có tiền** — trường hợp nghiêm trọng hơn.

ORD_004 bị phát hiện: `total_amount = 5M` nhưng không có item nào trong `Order_Items`.

ORD_004 còn có thêm bug `customer_id = C999` không tồn tại (Câu 5) — một đơn tích lũy nhiều lỗi chồng nhau.

Kết quả sau khi query (minh họa)

order_id	customer_id	total_amount	status
ORD_004	C999	5.000.000	PENDING

ORD_004: 5M nhưng không có sản phẩm nào. Thêm vào đó: C999 không tồn tại. Đây là đơn hàng 'ghost' — cần điều tra log xem được tạo như thế nào.

GÓC SOI LỖI CỦA TESTER

ORD_004 là ví dụ điển hình của **bug chồng bug**:

- (1) Câu 5: `customer_id = C999` không có trong `Customers`.
- (2) Câu 15: không có item nào trong `Order_Items`.
- (3) Câu 38: có `total_amount = 5M` nhưng không có gì để tính.

Khi tìm thấy loại đơn này, ưu tiên kiểm tra access log và payment log để xác định có transaction thật không.

39**Phát hiện tổng items vượt quá 1.5 lần `total_amount`****TÌNH HUỐNG**

Câu 11 bắt mọi đơn có tổng items khác `total_amount` dù chỉ 1 đồng. Câu này dùng ngưỡng $1.5\times$ để bắt những trường hợp **lệch bất thường lớn**: tổng items cao hơn `total_amount` gấp rưỡi là dấu hiệu dữ liệu sai ở mức độ không thể do làm tròn hay phí vận chuyển.

Bảng Orders — dòng đỏ: `total_amount` bất thường so với tổng items:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24

CÂU LỆNH SQL

```
SELECT o.order_id,
       o.total_amount,
       SUM(oi.quantity * oi.price) AS tinh_tu_items
FROM   Orders o
JOIN   Order_Items oi
      ON o.order_id = oi.order_id
GROUP BY o.order_id, o.total_amount
HAVING SUM(oi.quantity * oi.price)
       > o.total_amount * 1.5;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o JOIN Order_Items oi ON o.order_id = oi.order_id	INNER JOIN ghép đơn hàng với items. Đơn không có item (ORD_004) bị loại — dùng Câu 38 để phát hiện riêng trường hợp đó.
GROUP BY o.order_id, o.total_amount	Gom tất cả items của cùng một đơn để tính tổng.
HAVING SUM(oi.quantity * oi.price) > o.total_amount * 1.5	HAVING so sánh sau aggregate: tổng items > 1.5 lần total_amount là bất thường. Ngưỡng 1.5× để bỏ qua chênh lệch nhỏ do phí vận chuyển, discount.

Giải thích tổng thể

Câu 11 (= ANY difference) vs Câu 39 (> 1.5×): hai mức độ kiểm tra khác nhau.
ORD_001: tinh_tu_items = 62M, total_amount = 32M → 62M > 32M×1.5 = 48M → bị bắt.
ORD_002: tinh_tu_items = 31M, total_amount = 20M → 31M > 20M×1.5 = 30M → bị bắt.
Cả hai đều bị bắt: ORD_001 do item trùng, ORD_002 do Bug 1 (total_amount bị ghi thấp).

Kết quả sau khi query (minh họa)

order_id	total_amount	tinh_tu_items
ORD_001	32.000.000	62.000.000
ORD_002	20.000.000	31.000.000

2 đơn vượt ngưỡng 1.5×. ORD_001: 62M vs 32M (do item trùng). ORD_002: 31M vs 20M (do Bug 1 — total_amount bị ghi quá thấp). Hai nguyên nhân khác nhau, cùng một câu phát hiện.

GÓC SOI LỖI CỦA TESTER

Điều chỉnh ngưỡng tùy ngữ cảnh nghiệp vụ:

- (1) > 1.5×: bắt lệch lớn, bỏ qua discount/phí nhỏ (câu này).
- (2) != (bất kỳ lệch): bắt tất cả sai lệch dù nhỏ (Câu 11).
- (3) > 2×: chỉ bắt lệch nghiêm trọng nhất — ít false positive hơn.

Với hệ thống có discount, nên kết hợp thêm điều kiện loại trừ đơn có coupon trước khi áp ngưỡng.

40 Snapshot chất lượng dữ liệu: đếm dòng bốn bảng

TÌNH HUỐNG

Trước khi bắt đầu một vòng kiểm thử mới, QA cần biết nhanh: mỗi bảng có bao nhiêu dòng? Số dòng bất thường — quá ít hoặc quá nhiều so với sprint trước — là dấu hiệu sớm của migration lỗi, seed data thiếu, hoặc test cleanup chưa chạy.

Bốn bảng trong ecommerce_test — câu UNION ALL tổng hợp tất cả trong một lần:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24

CÂU LỆNH SQL


```
SELECT 'Customers' AS bang, COUNT(*) AS so_dong
FROM Customers
UNION ALL
SELECT 'Products', COUNT(*) FROM Products
UNION ALL
SELECT 'Orders', COUNT(*) FROM Orders
UNION ALL
SELECT 'Order_Items', COUNT(*) FROM Order_Items;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

SELECT 'Customers' AS bang, COUNT(*) AS so_dong FROM Customers	Mỗi khối SELECT trả về 1 dòng: tên bảng (hardcode) và COUNT(*) — tổng số dòng trong bảng đó.
UNION ALL	UNION ALL ghép các kết quả theo chiều dọc mà không loại dòng trùng. Dùng UNION ALL (không phải UNION) vì không cần dedup — mỗi bảng đã là dòng khác nhau, UNION thuần sẽ chậm hơn vô ích.

Giải thích tổng thể

UNION ALL cho phép ghép kết quả từ nhiều câu SELECT thành một bảng duy nhất.

Điều kiện để UNION ALL hoạt động: các SELECT phải có cùng số cột và kiểu dữ liệu tương thích.

Snapshot này cho thấy: Customers=10, Products=7, Orders=4, Order_Items=6.

Số dòng Order_Items (6) ít hơn đáng kể (4 orders × nhiều items) — dấu hiệu của orders rỗng và items trùng.

Kết quả sau khi query (minh họa)

bang	so_dong
Customers	10
Products	7
Orders	4
Order_Items	6

4 bảng, 27 dòng tổng cộng. Order_Items chỉ có 6 dòng cho 3 đơn có items (ORD_004 rỗng) — và 1 trong 6 dòng là item trùng (item_id=7).

GÓC SOI LỖI CỦA TESTER

Lưu ý quan trọng về **UNION vs UNION ALL**:

(1) **UNION ALL**: giữ tất cả dòng kể cả trùng — nhanh hơn vì không cần sort/dedup.

(2) **UNION**: loại dòng trùng — chậm hơn, nhưng cần thiết khi muốn distinct.

Câu này dùng UNION ALL vì tên bảng là chuỗi khác nhau — không có dòng nào trùng.

Mở rộng snapshot: thêm WHERE để đếm dòng theo điều kiện (vd: đếm Customers ACTIVE, Orders COMPLETED).

PHẦN 5 — Audit, log và dấu vết

Cột thời gian và mối quan hệ giữa các bảng là nơi kể lại lịch sử dữ liệu. Khi chúng mâu thuẫn — sản phẩm bị xóa vẫn còn trong đơn, khách hàng không tồn tại vẫn có giao dịch — đó là dấu hiệu của bug logic hoặc lỗi hỏng dữ liệu.

41 Tìm items trong đơn hàng trở tới sản phẩm không tồn tại

TÌNH HUỐNG

Câu 5 đã phát hiện ORD_004 trở tới C999 không tồn tại trong Customers. Câu này kiểm tra tương tự cho tầng dưới: Order_Items có item nào trở tới product_id không tồn tại trong Products không? Nếu có, đó là dấu hiệu sản phẩm bị xóa sau khi đã được đặt mua.

Bảng Order_Items — kiểm tra product_id có tồn tại trong Products không:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000

CÂU LỆNH SQL

```
SELECT oi.item_id,
       oi.order_id,
       oi.product_id
FROM   Order_Items oi
LEFT JOIN Products p
      ON oi.product_id = p.product_id
WHERE  p.product_id IS NULL;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items oi LEFT JOIN Products p ON oi.product_id = p.product_id	LEFT JOIN giữ tất cả items. Item có product_id tồn tại → p.product_id có giá trị. Item trở tới sản phẩm không tồn tại → p.product_id = NULL.
WHERE p.product_id IS NULL	Lọc ra những item không khớp được với sản phẩm nào — đây là anti-join pattern : dùng LEFT JOIN + IS NULL để tìm bản ghi bị mồ côi.
SELECT oi.item_id, oi.order_id, oi.product_id	Chiếu đủ để QA xác định item nào, thuộc đơn nào, và product_id nào không còn hợp lệ.

Giải thích tổng thể

Đây là ứng dụng tiếp theo của **anti-join pattern** — kỹ thuật lần đầu dùng ở Câu 5.
Câu 5: Orders → Customers (phát hiện ORD_004/C999).
Câu 41: Order_Items → Products (kiểm tra sản phẩm trong từng dòng đơn).
Kết quả rỗng trong data mẫu — tốt: mọi item đều trở tới sản phẩm hợp lệ.
Trên production, câu này quan trọng nếu hệ thống cho phép xóa sản phẩm mà không kiểm tra xem sản phẩm đó đang có trong đơn hàng nào.

Kết quả sau khi query (minh họa)

item_id	order_id	product_id
Kết quả rỗng — tất cả product_id trong Order_Items đều tồn tại trong Products. Chạy lại sau mỗi lần thực hiện thao tác xóa sản phẩm (soft-delete hay hard-delete).		

GÓC SOI LỖI CỦA TESTER

- Anti-join có hai cách viết — cùng kết quả, tốc độ khác nhau:
- (1) **LEFT JOIN ... WHERE IS NULL** (câu này): thường nhanh hơn trên MySQL vì optimizer có thể dùng index hiệu quả.
 - (2) **WHERE product_id NOT IN (SELECT product_id FROM Products)**: nguy hiểm nếu subquery trả về NULL — NOT IN với NULL luôn cho kết quả rỗng.
 - (3) **WHERE NOT EXISTS (...)**: an toàn với NULL, tốc độ tương đương LEFT JOIN.

42 Tìm sản phẩm chưa bao giờ được đặt mua (dùng NOT EXISTS)

TÌNH HUỐNG

Câu 16 đã dùng LEFT JOIN + IS NULL để tìm sản phẩm chưa bán. Câu này giải quyết cùng bài toán nhưng dùng **NOT EXISTS** — cú pháp đọc tự nhiên hơn và an toàn hơn khi cột JOIN có thể NULL. QA nên biết cả hai cách để đọc hiểu SQL do đội dev viết.

Bảng Products — dòng đỏ: sản phẩm chưa xuất hiện trong Order_Items:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
SELECT p.product_id,
       p.product_name
FROM   Products p
WHERE  NOT EXISTS (
    SELECT 1
    FROM    Order_Items oi
    WHERE   oi.product_id = p.product_id
);
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Products p	MySQL tải toàn bộ bảng Products — sẽ kiểm tra từng sản phẩm một.
WHERE NOT EXISTS (SELECT 1 FROM Order_Items oi WHERE oi.product_id = p.product_id)	NOT EXISTS chạy subquery tương quan: với mỗi sản phẩm p, MySQL kiểm tra xem Order_Items có dòng nào có product_id khớp không. Nếu không → NOT EXISTS = TRUE → giữ dòng này. SELECT 1 : không cần lấy dữ liệu thật, chỉ kiểm tra sự tồn tại.
SELECT p.product_id, p.product_name	Chiếu ID và tên để QA xác định sản phẩm nào chưa có test coverage.

Giải thích tổng thể
NOT EXISTS vs LEFT JOIN + IS NULL — hai cách viết cùng kết quả, khác cú pháp:
Câu 16: LEFT JOIN ... WHERE oi.product_id IS NULL — ngắn hơn, phổ biến hơn.
Câu 42: WHERE NOT EXISTS (...) — đọc tự nhiên hơn ('sản phẩm mà KHÔNG TỒN TẠI trong đơn nào'), an toàn hơn khi cột JOIN có thể mang giá trị NULL thật.
Kết quả giống nhau: PROD_005, PROD_006, PROD_007 chưa bán.

Ket qua sau khi query (minh hoa)

product_id	product_name
PROD_005	Ban phim co Logitech
PROD_006	Loa Bluetooth JBL
PROD_007	Chuot gaming Razer

3 sản phẩm chưa có trong đơn nào: PROD_005 (tên trùng), PROD_006 (price NULL), PROD_007 (stock NULL). Kết quả y hệt Câu 16.

GÓC SOI LỖI CỦA TESTER

Ba cách viết anti-join — chọn theo ngữ cảnh:

- (1) **LEFT JOIN ... WHERE IS NULL** (Câu 16): ngắn nhất, optimizer MySQL xử lý tốt.
- (2) **WHERE NOT EXISTS** (Câu 42): đọc tự nhiên nhất, an toàn với nullable FK.
- (3) **WHERE product_id NOT IN (SELECT ...)**: cẩn thận — nếu subquery trả về NULL, toàn bộ kết quả sẽ rỗng vì 'x NOT IN (NULL, ...)' luôn UNKNOWN (không phải TRUE).

43 Tìm khách hàng chưa bao giờ đặt hàng (dùng NOT EXISTS)

TÌNH HUỐNG

Câu 17 đã dùng LEFT JOIN + IS NULL để tìm khách chưa mua hàng. Câu này dùng **NOT EXISTS** — cùng kết quả, cú pháp khác. Nắm cả hai cách giúp QA đọc và viết SQL do nhiều người khác nhau viết trong dự án thực tế.

Bảng Customers — dòng đỏ: 7 khách hàng chưa có đơn hàng nào:

customer_id	customer_name	membership_tier	status
C001	Nguyen Van A	Silver	ACTIVE
C002	Tran Van B	Standard	ACTIVE

customer_id	customer_name	membership_tier	status
C003	Le Thi C	Gold	ACTIVE
C004	Khach Hang Ao Bug	Standard	ACTIVE
C005	Khach Hang Trung	Standard	ACTIVE
C006	Pham Van X	Standard	ACTIVE
C007	Nguyen Thi Y	Standard	ACTIVE
C008	Pham Van D	Gold	ACTIVE
C009	Nguyen Van A (2)	Silver	ACTIVE
C010	Khach Test VIP	VIP	ACTIVE

CÂU LỆNH SQL

```
SELECT c.customer_id,
       c.customer_name
FROM   Customers c
WHERE  NOT EXISTS (
    SELECT 1
    FROM    Orders o
    WHERE   o.customer_id = c.customer_id
);
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Customers c	MySQL tải toàn bộ bảng Customers — sẽ kiểm tra từng khách một.
WHERE NOT EXISTS (SELECT 1 FROM Orders o WHERE o.customer_id = c.customer_id)	NOT EXISTS: với mỗi khách c, MySQL chạy subquery tìm đơn hàng có customer_id khớp. Nếu không có đơn nào → NOT EXISTS = TRUE → khách này được đưa vào kết quả.
SELECT c.customer_id, c.customer_name	Chiếu ID và tên để QA xác định tài khoản dùng test luồng chưa có lịch sử.

Giải thích tổng thể

Câu 17 (LEFT JOIN IS NULL) và Câu 43 (NOT EXISTS): cùng kết quả, hai cú pháp.
Câu 42 và 43 tạo thành bộ đôi NOT EXISTS:
Câu 42: Products không có trong Order_Items → sản phẩm chưa bán.
Câu 43: Customers không có trong Orders → khách chưa mua.
7/10 khách chưa đặt đơn — tỷ lệ cao phản ánh data mẫu nhỏ, nhưng kỹ thuật NOT EXISTS áp dụng y hệt trên production với hàng triệu bản ghi.

Ket qua sau khi query (minh hoa)

customer_id	customer_name
C004	Khach Hang Ao Bug
C005	Khach Hang Trung
C006	Pham Van X
C007	Nguyen Thi Y
C008	Pham Van D

customer_id	customer_name
C009	Nguyen Van A (2)
C010	Khach Test VIP

7 khách chưa có đơn — kết quả y hệt Câu 17. Dùng danh sách này để test luồng: khách mới, giỏ hàng trống, 'chưa có đơn hàng nào'.

GÓC SOI LỖI CỦA TESTER

NOT EXISTS thường được optimizer MySQL xử lý hiệu quả vì dừng ngay khi tìm thấy 1 dòng khớp. Để kiểm tra khách chưa mua **trong 30 ngày gần đây** (không phải chưa từng mua), phải dùng LEFT JOIN thay NOT EXISTS:

LEFT JOIN Orders o

ON c.customer_id = o.customer_id

AND o.order_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY)

WHERE o.customer_id IS NULL

NOT EXISTS không làm được điều này vì không có chỗ đặt điều kiện ngày.

44 Phát hiện item_id bị nhảy — dấu vết của bản ghi bị xóa

TÌNH HUỐNG

item_id là khóa chính tự tăng (AUTO_INCREMENT). Nếu dãy số bị nhảy — ví dụ tồn tại 1, 2, 4 nhưng không có 3 — có nghĩa là một bản ghi đã từng tồn tại và bị xóa. Đây là kỹ thuật audit đơn giản để phát hiện dữ liệu bị xóa không có log.

Bảng Order_Items — item_id=3 bị thiếu trong dãy 1→7:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000

CÂU LỆNH SQL

```
SELECT a.item_id + 1      AS id_bi_mat
FROM   Order_Items a
LEFT JOIN Order_Items b
      ON b.item_id = a.item_id + 1
WHERE  b.item_id IS NULL
AND    a.item_id <
      (SELECT MAX(item_id)
       FROM Order_Items);
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

```
FROM Order_Items a
LEFT JOIN Order_Items b
      ON b.item_id = a.item_id + 1
```

Self-join: bảng tự join với chính nó. Alias **a** là bản ghi gốc, **b** là bản ghi tiếp theo liền kề. Nếu **b.item_id = NULL** → không có bản ghi nào có **id = a.item_id + 1**.

<pre>WHERE b.item_id IS NULL AND a.item_id < (SELECT MAX(item_id) FROM Order_Items)</pre>	Hai điều kiện kết hợp: tiếp theo bị thiếu (IS NULL) VÀ chưa phải bản ghi cuối (< MAX). Bỏ điều kiện MAX sẽ trả thêm id = MAX+1, không phải gap.
<pre>SELECT a.item_id + 1 AS id_bi_mat</pre>	Kết quả là id bị thiếu — tức là a.item_id + 1.

Giải thích tổng thể

Self-join là kỹ thuật dùng một bảng join với chính nó để phát hiện mối quan hệ giữa các dòng trong cùng bảng.

Logic: với mỗi item_id=n, kiểm tra xem n+1 có tồn tại không. Nếu không → n+1 là gap.

item_id=2 tồn tại nhưng item_id=3 không → trả về id_bi_mat=3.

item_id=7 là MAX nên không kiểm tra tiếp — 8 không phải gap, chỉ là chưa có.

Kết quả sau khi query (minh họa)

id_bi_mat
3

item_id=3 bị thiếu trong dãy 1→7. Một item đã từng tồn tại và bị xóa, hoặc INSERT thất bại và AUTO_INCREMENT vẫn tiếp tục tăng.

GÓC SOI LỖI CỦA TESTER

Gap trong AUTO_INCREMENT không phải lúc nào cũng là bug:

- (1) **INSERT thất bại**: transaction rollback, nhưng AUTO_INCREMENT không rollback lại — tạo gap bình thường.
- (2) **Xóa dữ liệu**: hard DELETE không để lại dấu vết gì khác.
- (3) **Bulk insert lỗi giữa chừng**: một số ID đã được cấp nhưng bản ghi tương ứng không được lưu. Để biết nguyên nhân chính xác, cần có bảng audit log riêng.

45**Phân tích đơn hàng bị hủy — khách nào hay hủy nhất****TÌNH HUỐNG**

Đơn bị hủy không chỉ là vấn đề kinh doanh — còn là tín hiệu kỹ thuật. Nếu một khách hàng cụ thể có tỷ lệ hủy cao bất thường, có thể luồng thanh toán gặp lỗi chỉ với profile đó, hoặc dữ liệu test chưa được dọn sạch sau sprint.

Bảng Orders — dòng đỏ: ORD_003 có status = CANCELLED:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24

CÂU LỆNH SQL

```
SELECT c.customer_id,
       c.customer_name,
       COUNT(o.order_id) AS so_don_huy
FROM   Orders o
JOIN   Customers c
      ON o.customer_id = c.customer_id
WHERE  o.status = 'CANCELLED'
GROUP BY c.customer_id, c.customer_name
ORDER BY so_don_huy DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders o JOIN Customers c ON o.customer_id = c.customer_id	INNER JOIN: chỉ ghép đơn có customer_id hợp lệ. ORD_004 (C999) bị loại khỏi kết quả vì C999 không có trong Customers.
WHERE o.status = 'CANCELLED'	Lọc chỉ giữ đơn bị hủy trước khi group.
GROUP BY c.customer_id, c.customer_name ORDER BY so_don_huy DESC	Gom theo khách hàng và sắp xếp giảm dần — khách hủy nhiều nhất lên đầu.

Giải thích tổng thể

Kỹ thuật **WHERE** → **GROUP BY** → **ORDER BY** là chuỗi chuẩn để phân tích theo nhóm:

- (1) WHERE lọc loại đơn cần phân tích trước.
- (2) GROUP BY gom về từng khách hàng.
- (3) ORDER BY DESC đặt trường hợp đáng ngờ nhất lên đầu.

Data mẫu nhỏ nên chỉ có C003/1 đơn — trên production, câu này hiệu quả nhất khi chạy kèm HAVING COUNT(*) > N để lọc noise.

Kết quả sau khi query (minh họa)

customer_id	customer_name	so_don_huy
C003	Le Thi C	1

C003 hủy 1 đơn (ORD_003). Với data mẫu nhỏ, không kết luận được pattern — câu này cho thấy kỹ thuật; giá trị thực sự khi chạy trên production đủ lớn.

GÓC SOI LỖI CỦA TESTER

Mở rộng câu này để phân tích sâu hơn:

- (1) Thêm **HAVING COUNT(*) > 2**: chỉ hiện khách hủy nhiều bất thường.
- (2) Thêm **tỷ lệ**: số đơn hủy / tổng số đơn của khách — khách có 1 đơn và hủy 1 đơn = 100% hủy, đáng ngờ hơn khách có 10 đơn hủy 1.
- (3) Kết hợp **order_date**: xem các đơn hủy có tập trung trong một khoảng thời gian cụ thể không — có thể liên quan đến release lỗi.

PHẦN 6 — Truy vấn nâng cao cho QA

Năm câu lệnh cuối dùng tới window function, CTE và UNION. Chúng giải quyết những bài toán kiểm thử khó: lấy bản ghi mới nhất mỗi nhóm, phân hạng khách hàng, tổng hợp nhiều loại lỗi trong một báo cáo duy nhất.

46

Dùng ROW_NUMBER() phát hiện item trùng trong cùng một đơn

TÌNH HUỐNG

Câu 2 đã phát hiện item trùng bằng GROUP BY + HAVING COUNT(*) > 1 — cho biết nhóm nào bị trùng nhưng không chỉ ra dòng nào là bản gốc, dòng nào là bản thừa. Câu này dùng **window function ROW_NUMBER()** để đánh số thứ tự từng item trong cùng nhóm (order_id + product_id). Bất kỳ item nào có số thứ tự > 1 là bản ghi trùng — và ta có thể thấy chính xác item nào là lần xuất hiện đầu tiên, lần nào là trùng.

Bảng Order_Items — dòng đỏ: item 1 và item 7 cùng ORD_001/PROD_001:

item_id	order_id	product_id	quantity	price
1	ORD_001	PROD_001	1	30.000.000
2	ORD_001	PROD_002	1	2.000.000
4	ORD_002	PROD_001	1	30.000.000
5	ORD_002	PROD_004	1	1.000.000
6	ORD_003	PROD_003	1	8.000.000
7	ORD_001	PROD_001	1	30.000.000

CÂU LỆNH SQL

```
SELECT item_id,
       order_id,
       product_id,
       ROW_NUMBER() OVER (
         PARTITION BY order_id, product_id
         ORDER BY item_id
       ) AS so_lan_trong_don
FROM   Order_Items
ORDER BY order_id, product_id, item_id;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Order_Items	MySQL tải toàn bộ bảng Order_Items.
ROW_NUMBER() OVER (PARTITION BY order_id, product_id ORDER BY item_id AS so_lan_trong_don	ROW_NUMBER() là window function — tính toán trên một 'cửa sổ' dữ liệu mà không gộp các dòng lại. PARTITION BY chia dữ liệu thành nhóm (như GROUP BY nhưng giữ nguyên từng dòng). ORDER BY item_id xác định thứ tự đánh số trong mỗi nhóm.

ORDER BY order_id, product_id, item_id	Sắp xếp kết quả để dễ quan sát — các item cùng nhóm (order+product) xếp liền nhau.
---	--

Giải thích tổng thể

Window function khác GROUP BY ở chỗ: GROUP BY gộp nhiều dòng thành 1, còn window function giữ nguyên tất cả dòng và thêm một cột tính toán.
PARTITION BY order_id, product_id tạo nhóm cho mỗi cặp (đơn hàng + sản phẩm).
Trong nhóm ORD_001/PROD_001: item_id=1 được đánh số 1, item_id=7 được đánh số 2.
Bất kỳ dòng nào có **so_lan_trong_don > 1** là bản ghi trùng trong cùng đơn.

Ket qua sau khi query (minh hoa)

item_id	order_id	product_id	so_lan_trong_don
1	ORD_001	PROD_001	1
7	ORD_001	PROD_001	2
2	ORD_001	PROD_002	1
4	ORD_002	PROD_001	1
5	ORD_002	PROD_004	1
6	ORD_003	PROD_003	1

Item_id=7 có so_lan_trong_don=2 — đây là bản ghi trùng của item_id=1 trong cùng ORD_001/PROD_001. Lọc WHERE so_lan_trong_don > 1 để chỉ xem bản trùng.

GÓC SOI LỖI CỦA TESTER

Ứng dụng thực tế của ROW_NUMBER() trong QA:

(1) **Phát hiện trùng**: WHERE so_lan > 1 → xem chính xác dòng nào là trùng.

(2) **Lấy bản ghi mới nhất**: PARTITION BY customer_id ORDER BY order_date DESC → lấy dòng số 1 của mỗi khách = đơn mới nhất.

(3) **Phân trang logic**: đánh số thứ tự trong nhóm để lấy top-N mỗi nhóm.

ROW_NUMBER yêu cầu MySQL 8.0+. Kiểm tra phiên bản: **SELECT VERSION();**

47

Dùng CTE + RANK() xếp hạng sản phẩm bán chạy

TÌNH HUỐNG

Kết hợp hai kỹ thuật nâng cao: **CTE** (Common Table Expression) để tính doanh số trung gian, sau đó **RANK()** để xếp hạng. Đây là cách viết SQL rõ ràng hơn nhiều so với subquery lồng nhau — đặc biệt hữu ích khi kiểm thử hệ thống báo cáo.

Bảng Products — PROD_001 xếp hạng 1 nhưng doanh số bị thổi phồng do item trùng:

product_id	product_name	category	price	stock
PROD_001	iPhone 15 Pro Max	Dien thoai	30.000.000	50
PROD_002	Ban phim co Logitech	Phu kien	2.000.000	100
PROD_003	Tai nghe Sony WH-1000XM5	Phu kien	8.000.000	-5
PROD_004	Sac du phong Anker	Phu kien	1.000.000	20
PROD_005	Ban phim co Logitech	Phu kien	2.000.000	30
PROD_006	Loa Bluetooth JBL	Phu kien	(NULL)	10

product_id	product_name	category	price	stock
PROD_007	Chuot gaming Razer	Phu kien	1.500.000	(NULL)

CÂU LỆNH SQL

```
WITH doanh_so AS (  
  SELECT p.product_id,  
         p.product_name,  
         SUM(oi.quantity * oi.price)  
           AS tong_doanh_so  
  FROM   Products p  
  JOIN   Order_Items oi  
        ON p.product_id = oi.product_id  
  GROUP BY p.product_id, p.product_name  
)  
SELECT product_id,  
       product_name,  
       tong_doanh_so,  
       RANK() OVER  
         (ORDER BY tong_doanh_so DESC)  
         AS hang  
FROM   doanh_so;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

WITH doanh_so AS (SELECT ..., SUM(...) FROM Products JOIN Order_Items GROUP BY ...)	CTE (WITH ... AS): đặt tên cho một kết quả trung gian. Câu SELECT bên trong tính tổng doanh số — giống Câu 22 nhưng được đặt tên 'doanh_so' để tái sử dụng bên dưới.
RANK() OVER (ORDER BY tong_doanh_so DESC) AS hang	RANK(): gán số thứ hạng theo thứ tự. Khác ROW_NUMBER: nếu hai sản phẩm bằng doanh số, cả hai cùng nhận hạng 1 và hạng 2 bị bỏ qua (1, 1, 3...).
FROM doanh_so	Tham chiếu tới CTE như một bảng thông thường — đây là điểm mạnh của CTE: viết một lần, dùng nhiều chỗ.

Giải thích tổng thể

CTE giúp viết SQL theo kiểu 'từng bước' thay vì lồng subquery:

Bước 1 (WITH): tính tổng doanh số từng sản phẩm → lưu vào 'doanh_so'.

Bước 2 (SELECT chính): lấy từ 'doanh_so', thêm cột RANK().

PROD_001 xếp hạng 1 với 90M nhưng con số này bị thổi phồng do item trùng (Câu 10/46) — doanh số thực chỉ khoảng 60M.

Ket qua sau khi query (minh hoa)

product_id	product_name	tong_doanh_so	hang
PROD_001	iPhone 15 Pro Max	90.000.000	1
PROD_003	Tai nghe Sony WH-1000XM5	8.000.000	2
PROD_002	Ban phim co Logitech	2.000.000	3
PROD_004	Sac du phong Anker	1.000.000	4

4 sản phẩm có doanh số (PROD_005, 006, 007 chưa bán — Câu 42). PROD_001 dẫn đầu với 90M nhưng bị phóng đại do item_id=7 trùng lặp.

GÓC SOI LỖI CỦA TESTER

Ba loại xếp hạng window function — chọn đúng theo nghiệp vụ:

- (1) **ROW_NUMBER()**: luôn số liên tiếp, không cùng hạng — 1, 2, 3, 4.
 - (2) **RANK()**: cùng điểm = cùng hạng, bỏ hạng tiếp theo — 1, 1, 3, 4.
 - (3) **DENSE_RANK()**: cùng điểm = cùng hạng, không bỏ hạng — 1, 1, 2, 3.
- Với báo cáo doanh số, RANK() thường phù hợp hơn ROW_NUMBER().

48 Dùng CTE lồng nhau tìm khách chi tiêu trên mức trung bình**TÌNH HUỐNG**

Câu 47 dùng CTE một lớp. Câu này đi thêm một bước: **tái sử dụng CTE trong subquery** ngay bên trong câu SELECT chính — tính tổng chi tiêu từng khách, rồi lọc những người vượt trung bình của chính tập đó. Không dùng CTE, câu này phải viết subquery lồng 3 cấp.

Bảng Customers — C001 là khách duy nhất chi tiêu trên mức trung bình COMPLETED:

customer_id	customer_name	membership_tier	status
C001	Nguyen Van A	Silver	ACTIVE
C002	Tran Van B	Standard	ACTIVE
C003	Le Thi C	Gold	ACTIVE
C004	Khach Hang Ao Bug	Standard	ACTIVE
C005	Khach Hang Trung	Standard	ACTIVE
C006	Pham Van X	Standard	ACTIVE
C007	Nguyen Thi Y	Standard	ACTIVE
C008	Pham Van D	Gold	ACTIVE
C009	Nguyen Van A (2)	Silver	ACTIVE
C010	Khach Test VIP	VIP	ACTIVE

CÂU LỆNH SQL

```
WITH tong_chi AS (
  SELECT c.customer_id,
         c.customer_name,
         SUM(o.total_amount) AS tong_da_mua
  FROM   Customers c
  JOIN   Orders o
        ON c.customer_id = o.customer_id
  WHERE  o.status = 'COMPLETED'
  GROUP BY c.customer_id, c.customer_name
)
SELECT customer_id,
       customer_name,
       tong_da_mua
FROM   tong_chi
WHERE  tong_da_mua >
      (SELECT AVG(tong_da_mua)
       FROM tong_chi)
ORDER BY tong_da_mua DESC;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

WITH tong_chi AS (... WHERE status = 'COMPLETED' GROUP BY customer_id ...)	CTE tính tổng chi tiêu từng khách, chỉ tính đơn COMPLETED. Lọc WHERE bên trong CTE — không tính đơn CANCELLED hay PENDING.
WHERE tong_da_mua > (SELECT AVG(tong_da_mua) FROM tong_chi)	Điểm mạnh của CTE: tái sử dụng 'tong_chi' trong subquery ngay bên trong mệnh đề WHERE. Không CTE, ta phải tính lại toàn bộ GROUP BY một lần nữa trong subquery.
ORDER BY tong_da_mua DESC	Sắp xếp giảm dần — khách chi nhiều nhất lên đầu.

Giải thích tổng thể

CTE 'tong_chi' được dùng **hai lần** trong câu lệnh — đây là lý do chính để dùng CTE:

- (1) FROM tong_chi: lấy danh sách từng khách và tổng chi tiêu.
 - (2) SELECT AVG(tong_da_mua) FROM tong_chi: tính trung bình từ cùng tập đó.
- Tổng COMPLETED: C001=32M, C002=20M → trung bình = 26M → chỉ C001 (32M) vượt qua.

Kết quả sau khi query (minh họa)

customer_id	customer_name	tong_da_mua
C001	Nguyen Van A	32.000.000

Chỉ C001 (32M) vượt mức trung bình 26M. C002 có 20M < 26M nên bị loại. Đây là khách VIP tiềm năng — dữ liệu test nên cover luồng upgrade membership.

GÓC SOI LỖI CỦA TESTER

CTE vs Subquery — khi nào dùng cái nào:

- (1) **Dùng CTE** khi cần tái sử dụng kết quả trung gian nhiều lần, hoặc khi logic chia thành nhiều bước rõ ràng.
- (2) **Dùng subquery** khi logic đơn giản và chỉ dùng một lần.
- (3) **CTE đệ quy** (WITH RECURSIVE): dùng cho cấu trúc cây (category cha/con, cây tổ chức) — MySQL 8.0+ hỗ trợ.

49 Tính doanh thu tích lũy theo thời gian với SUM() OVER()

TÌNH HUỐNG

Báo cáo tài chính thường yêu cầu 'running total' — tổng cộng dồn theo thời gian. Không có window function, ta phải dùng self-join hoặc subquery tương quan rất phức tạp. **SUM() OVER()** giải quyết trong một câu lệnh đơn giản, và QA dùng nó để kiểm tra xem hệ thống báo cáo có tính đúng không.

Bảng Orders — doanh thu tích lũy tính theo order_date tăng dần:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24

CÂU LỆNH SQL

```
SELECT order_id,
       customer_id,
       order_date,
       total_amount,
       SUM(total_amount) OVER (
         ORDER BY order_date
         ROWS BETWEEN UNBOUNDED PRECEDING
         AND CURRENT ROW
       ) AS luy_ke
FROM   Orders
ORDER BY order_date;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

FROM Orders	MySQL tải toàn bộ bảng Orders — tất cả 4 đơn, kể cả CANCELLED và PENDING.
SUM(total_amount) OVER (ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS luy_ke	SUM() OVER(): cộng dồn total_amount theo thứ tự order_date. ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW: cửa sổ tính từ dòng đầu tiên đến dòng hiện tại. Có thể viết ngắn hơn: SUM(...) OVER (ORDER BY order_date) — cùng kết quả.
ORDER BY order_date	Sắp xếp kết quả cuối cùng theo ngày — đảm bảo luy_ke tăng dần theo thời gian khi đọc từ trên xuống.

Giải thích tổng thể

SUM() OVER() tính tổng tích lũy mà không gộp dòng — giữ nguyên từng đơn hàng.
ORD_001 (32M) → luy_ke = 32M. ORD_002 (20M) → luy_ke = 52M. ORD_003 (8M, CANCELLED) → luy_ke = 60M. ORD_004 (5M, C999) → luy_ke = 65M.
Lưu ý: câu này cộng tất cả đơn kể cả CANCELLED và PENDING — luy_ke thực tế (chỉ COMPLETED) là 52M, không phải 65M.

Kết quả sau khi query (minh họa)

order_id	customer_id	order_date	total_amount	luy_ke
ORD_001	C001	2026-06-20	32.000.000	32.000.000
ORD_002	C002	2026-06-22	20.000.000	52.000.000
ORD_003	C003	2026-06-23	8.000.000	60.000.000
ORD_004	C999	2026-06-24	5.000.000	65.000.000

luy_ke = 65M nhưng đây là tổng thô gồm cả đơn CANCELLED (ORD_003/8M) và đơn lỗi (ORD_004/5M). Doanh thu thực COMPLETED: 52M.

GÓC SOI LỖI CỦA TESTER

Để tính running total chỉ của đơn COMPLETED:

SUM(CASE WHEN status = 'COMPLETED'
THEN total_amount ELSE 0 END)
OVER (ORDER BY order_date) AS luy_ke_thuc

Kỹ thuật CASE WHEN bên trong window function cho phép tính tích lũy có điều kiện mà không cần WHERE lọc trước (vì WHERE loại bỏ dòng → mất ORDER BY liên tục theo thời gian).

50

Báo cáo tổng hợp: nhiều loại lỗi trong một câu UNION ALL

TÌNH HUỐNG

Câu cuối tổng hợp nhiều kiểm tra thành **một báo cáo duy nhất**: email trùng (Câu 6), tồn kho âm (Câu 12), khách hàng ma (Câu 5). QA dùng câu này như một 'health check' nhanh — chạy một lần, thấy ngay hệ thống đang có bao nhiêu loại lỗi tồn đọng.

Tổng hợp ba loại lỗi từ Customers, Products, Orders trong một lần chạy:

order_id	customer_id	total_amount	status	order_date
ORD_001	C001	32.000.000	COMPLETED	2026-06-20
ORD_002	C002	20.000.000	COMPLETED	2026-06-22
ORD_003	C003	8.000.000	CANCELLED	2026-06-23
ORD_004	C999	5.000.000	PENDING	2026-06-24

CÂU LỆNH SQL

```
SELECT 'Email trùng' AS loại_loi,
       customer_id AS doi_tuong,
       'Customers' AS bang
FROM   Customers
WHERE  email IN (
  SELECT email FROM Customers
  WHERE email IS NOT NULL
  AND   email != ''
  GROUP BY email
  HAVING COUNT(*) > 1)
UNION ALL
SELECT 'Ton kho am', product_id, 'Products'
FROM   Products WHERE stock < 0
UNION ALL
SELECT 'Khach hang ma', order_id, 'Orders'
FROM   Orders
WHERE  customer_id NOT IN
  (SELECT customer_id FROM Customers)
ORDER BY loại_loi;
```

PHÂN TÍCH TỪNG MỆNH ĐỀ SQL (theo thứ tự MySQL thực thi — FROM chạy trước SELECT)

SELECT 'Email trùng' AS loại_loi, customer_id, 'Customers' AS bang FROM Customers WHERE email IN (...)	Khối 1: tìm email trùng bằng subquery (tương tự Câu 6 nhưng nhúng vào đây). Cột 'Email trùng' là chuỗi cố định — giúp phân biệt loại lỗi trong kết quả gộp.
UNION ALL SELECT 'Ton kho am', ... UNION ALL SELECT 'Khach hang ma', ...	UNION ALL ghép ba khối SELECT thành một bảng. Điều kiện: cùng số cột, kiểu dữ liệu tương thích. Dùng UNION ALL thay UNION để không mất dòng trùng — các lỗi ở bảng khác nhau không bao giờ trùng.
ORDER BY loại_loi	Sắp xếp theo loại lỗi — nhóm các lỗi cùng loại lại gần nhau để dễ đọc báo cáo.

Giải thích tổng thể

Kết quả có 6 dòng — nhiều hơn dự kiến do một phát hiện bất ngờ:

Email trùng: C001, C004, C005, C009 — không chỉ C004 và C005.

Nguyên nhân: MySQL mặc định so sánh VARCHAR **không phân biệt hoa/thường** (collation utf8_general_ci). 'a.nguyen@email.com' (C001) và 'A.NGUYEN@EMAIL.COM' (C009) được coi là cùng email → cả hai bị bắt.

Đây là bug ẩn mà Câu 6 cũng đã bắt — cần xác nhận lại policy: hệ thống có phân biệt hoa thường trong email không?

Kết quả sau khi query (minh họa)

loai_loi	doi_tuong	bang
Email trùng	C001	Customers
Email trùng	C004	Customers
Email trùng	C005	Customers
Email trùng	C009	Customers
Khách hàng ma	ORD_004	Orders
Tồn kho âm	PROD_003	Products

6 lỗi từ 3 loại: 4 email trùng (C001+C009 vì MySQL case-insensitive), 1 đơn mờ côi (ORD_004), 1 tồn kho âm (PROD_003).
Chạy câu này mỗi ngày để theo dõi dữ liệu sạch hay không.

GÓC SOI LỖI CỦA TESTER

Mở rộng health check: thêm loại lỗi mới bằng cách thêm UNION ALL:

UNION ALL

SELECT 'Gia NULL', product_id, 'Products'

FROM Products WHERE price IS NULL

Để báo cáo có thêm cột 'so_luong' đếm bao nhiêu lỗi mỗi loại, bọc toàn bộ UNION ALL vào một CTE rồi GROUP BY loại_loi bên ngoài.

Tóm lại

SQL không thay thế tư duy kiểm thử — nó khuếch đại tư duy ấy. Một câu lệnh chỉ mạnh khi đứng sau nó là một câu hỏi tốt: "Điều gì trong hệ thống này lẽ ra phải luôn đúng?". 50 câu lệnh ở đây là 50 câu hỏi như vậy, đã được viết thành truy vấn. Khi gặp một nghiệp vụ mới, hãy tự đặt câu hỏi của riêng bạn rồi dịch nó sang SQL theo đúng các mẫu trong sách.

Ba điều mang theo:

- Mọi truy vấn đối soát đều dựa trên một **bất biến** — tìm cái luôn đúng, rồi tìm cái vi phạm nó.
- Kết quả SQL là **danh sách cần điều tra**, không phải bản án. Luôn xác minh trước khi kết luận bug.
- Cẩn thận với **NULL**, **collation**, **kiểu số thực** và **khác biệt dialect** — đó là nơi bug ẩn nấp ngay trong chính câu lệnh của bạn.

— Biên soạn bởi MAI.tools · Ứng dụng AI vào công việc Kiểm thử · maiqai.com